

Operating Systems

PART 1 — OS FUNDAMENTALS AND SYSTEM CALLS

1.1 What an OS does

Four roles: **resource manager** (CPU, memory, I/O, files), **abstraction provider** (hides hardware detail), **control program** (prevents errors and improper use), **kernel** (the always-resident core).

1.2 Types of OS — all of them

Type	Defining property	Example
Batch	Jobs grouped, no interaction, operator-sequenced	Early IBM systems
Multiprogrammed	Several jobs in memory; CPU switches on I/O wait	—
Time-sharing / multitasking	CPU switches on time quantum ; interactive	UNIX
Real-time — hard	Deadline miss = system failure	Airbag, pacemaker
Real-time — soft	Deadline miss = degraded quality	Video streaming
Distributed	Multiple machines, single logical system	—
Network	Machines keep individual identity, share resources	—
Clustered	Tightly coupled machines with shared storage	—
Embedded	Fixed function, resource-constrained	Router firmware
Mobile	Power and memory optimised	Android, iOS

The distinction that gets tested: multiprogramming switches on I/O wait; multitasking switches on quantum expiry; multiprocessing means multiple CPUs. Three different words, three different meanings, constantly swapped in options.

1.3 Kernel architectures

Architecture	Services in kernel space	Trade-off
Monolithic	All	Fast (no message passing), poor fault isolation, hard to maintain — Linux, traditional UNIX
Microkernel	Minimum (IPC, scheduling, basic memory)	Reliable, extensible, slower due to message passing overhead — QNX, Minix, Mach
Hybrid	Mixed	Windows NT, macOS (XNU)
Exokernel	Almost nothing; exposes hardware	Research
Layered	Organised in strict layers	THE OS; clean but slow

Why microkernels are slower — a service call that was a function call in a monolithic kernel becomes a message round-trip through the kernel. This is the standard explanation question.

1.4 Mode switch vs context switch — do not confuse these

	Mode switch	Context switch
What changes	Privilege level only (user ↔ kernel)	The running process
Triggered by	System call, interrupt, trap	Scheduler decision, quantum expiry, blocking
Cost	Cheap	Expensive
Saves PCB?	No	Yes

Every context switch involves mode switches. **Not every mode switch is a context switch.** A system call that returns to the same process is a mode switch with no context switch.

1.5 Interrupts, traps and system calls

- **Interrupt** — asynchronous, hardware-generated (I/O completion, timer).
- **Trap / exception** — synchronous, software-generated (divide by zero, invalid memory access, deliberate `int` instruction).
- **System call** — a deliberate trap requesting a kernel service.

Dual-mode operation is enforced by a **mode bit** in hardware. Privileged instructions (I/O, interrupt handling, timer setting, halt) execute only in kernel mode.

1.6 System calls — five categories, with the full list

Category	Calls
Process control	fork, exec, wait, waitpid, exit, abort, kill, nice, getpid
File management	open, close, read, write, lseek, create, unlink, stat, chmod
Device management	ioctl, read, write, request, release
Information maintenance	getpid, alarm, sleep, time, getrusage
Communication	pipe, shmget, shmat, mmap, msgget, send, recv, socket, connect

The recurring trap: `printf`, `scanf`, `malloc`, `free`, `fopen`, `strcpy` are **library functions**, not system calls. Several of them *invoke* system calls internally (`printf` → `write`, `malloc` → `brk/mmap`), but they are not themselves system calls. Questions phrased “which of the following is NOT a system call” almost always answer to one of these.

1.7 What `read()` actually does — asked verbatim

Traps to the kernel → validates the file descriptor and buffer → checks the buffer cache → if the data is absent, issues device I/O and blocks the process → data is copied **from kernel space into the user-space buffer** → returns the byte count.

It does **not** run in user mode, does **not** copy directly from hardware to user space, and does **not** necessarily block until EOF. All three appear as distractors.

1.8 Booting

Power on → **BIOS/UEFI** firmware POST → reads the **bootstrap loader** from the MBR / boot block → loads the **kernel** into memory → kernel initialises → starts the first process (`init` / `systemd`, PID 1).

PART 2 — PROCESSES AND THREADS

2.1 Process concept

A **process** is a program in execution. A program is passive (a file on disk); a process is active (has a PC, registers, stack).

Memory layout of a process, bottom to top:

Section	Contents	Grows
Text	Executable code	Fixed
Data	Initialised and uninitialised globals/statics	Fixed
Heap	Dynamically allocated memory	Upward
Stack	Function frames, locals, return addresses	Downward

Heap and stack grow toward each other. **A stack-heap collision is a memory exhaustion condition** — sometimes asked.

2.2 Process states and the transition diagram

Five states: New → Ready → Running → Waiting/Blocked → Terminated.

Valid transitions:

From → To	Cause
New → Ready	Admitted
Ready → Running	Scheduler dispatch
Running → Ready	Preemption — quantum expiry or higher-priority arrival
Running → Waiting	I/O request, <code>wait()</code> , resource unavailable
Waiting → Ready	I/O completion, event occurred
Running → Terminated	<code>exit()</code>

Invalid transitions — the exam's favourite: - **Waiting → Running is IMPOSSIBLE.** A blocked process must re-enter Ready and be dispatched. - **Ready → Waiting is IMPOSSIBLE.** A process must be running to request I/O. - Ready → Terminated only via `kill`, not normal flow.

Two extra states in systems with swapping: *suspended ready* and *suspended blocked* (process swapped out to disk). Seven-state model. Asked in PSU papers.

2.3 Process Control Block — complete contents

Process ID and parent ID · process state · program counter · CPU registers · CPU scheduling information (priority, queue pointers, scheduling parameters) · memory management information (base and limit registers, page tables, segment tables) · accounting information (CPU used, time limits, account numbers) · I/O status information (open file table, allocated devices)

2.4 Schedulers — three of them

Scheduler	Also called	Moves processes	Frequency
Long-term	Job scheduler	New → Ready	Infrequent; controls degree of multiprogramming
Short-term	CPU scheduler	Ready → Running	Very frequent (ms)
Medium-term	Swapper	Ready/Blocked ↔ Suspended	As needed; reduces multiprogramming degree

The degree of multiprogramming is controlled by the long-term scheduler. Directly asked. Time-sharing systems often have no long-term scheduler at all.

Dispatcher — the module that actually hands the CPU over. **Dispatch latency** is the time to stop one process and start another.

2.5 Threads

A thread is a lightweight unit of execution within a process.

Shared across threads of a process	Private to each thread
Code (text) section	Stack
Data section	Registers
Open files / file descriptors	Program counter
Signals and signal handlers	Thread ID
Heap	Errno

Learn this split cold. “Which of the following is NOT shared between threads” is a guaranteed-appearance question, and the answer is always stack, registers, PC, or thread ID.

Benefits: responsiveness, resource sharing, economy (cheaper to create and switch), scalability on multiprocessors.

User-level vs kernel-level threads

	User-level	Kernel-level
Managed by	Thread library in user space	The OS kernel
Kernel awareness	None	Full
Switching cost	Very cheap (no mode switch)	Expensive
One thread blocks	Entire process blocks	Only that thread blocks
True parallelism on multicore	No	Yes

Multithreading models

- **Many-to-one** — many user threads → one kernel thread. Cheap; one blocking call blocks all; no multicore parallelism.
- **One-to-one** — each user thread → one kernel thread. True concurrency; creation overhead limits thread count. Linux, Windows.
- **Many-to-many** — m user threads multiplexed onto n kernel threads ($m \geq n$). Most flexible; hardest to implement.
- **Two-level** — many-to-many plus the option to bind a specific user thread to a kernel thread.

Thread-specific issues

- **fork() in a multithreaded process** — two variants: duplicate all threads, or duplicate only the calling thread. POSIX provides both.
- **Thread cancellation** — *asynchronous* (terminate immediately, may leave state inconsistent) vs *deferred* (thread checks a cancellation point and exits cleanly). Deferred is safer.
- **Thread pool** — pre-created threads serve requests; bounds thread count and avoids creation cost.

NUMERICAL TYPE 1 — fork() counting

Base rule: n sequential unconditional fork() calls → **2ⁿ total processes, 2ⁿ - 1 children.**

Return values: 0 in the child, child's PID in the parent, -1 on failure.

Case A — plain sequence

```
fork(); fork(); fork();
```

2³ = 8 processes, 7 children.

Case B — loop

```
for (i = 0; i < 5; i++) fork();
```

2⁵ = 32 processes, 31 children.

Case C — with a print after the forks

```
fork(); fork(); printf("A");
```

All 4 processes reach the printf → **"A" printed 4 times.**

Case D — print inside the loop

```
for (i = 0; i < 3; i++) { fork(); printf("A"); }
```

Count processes existing at each print. Iteration 1: 2 processes print → 2 A's. Iteration 2: 4 print → 4. Iteration 3: 8 print → 8. **Total 14 A's.**

General result: for n iterations with the print after the fork, total prints = $2^1 + 2^2 + \dots + 2^n = 2^{n+1} - 2$.

Case E — conditional on the return value

```
if (fork() == 0) fork();
```

Parent gets non-zero → skips. Child gets 0 → forks again. **3 processes total.**

Case F — conditional the other way

```
if (fork() != 0) fork();
```

Parent gets non-zero → forks again. Child gets 0 → skips. **3 processes total.** Same count, different tree shape.

Case G — fork() inside a short-circuit

```
fork() && fork();
```

&& evaluates the second fork() only if the first returned non-zero (parent). Original: first fork → non-zero → evaluates second fork → 2 processes from here. First child: returns 0 → short-circuits, no second fork. Total = **3 processes.**

Case H — fork() with OR

```
fork() || fork();
```

`||` evaluates the second only if the first returned 0 (child). Original: non-zero → short-circuits. Child: 0 → forks. Total = **3 processes**.

Method for any variant: draw the process tree. Mark each node with what `fork()` returned there. Never try to memorise formulas beyond the plain 2ⁿ case.

NUMERICAL TYPE 2 — context switch count and overhead

Worked. Three processes, all arriving at $t = 0$, bursts 4, 3, 5. Round Robin, quantum 2, context switch overhead 1 ms.

Time	Activity
0-2	P1 (rem 2)
2-3	context switch
3-5	P2 (rem 1)
5-6	context switch
6-8	P3 (rem 3)
8-9	context switch
9-11	P1 done
11-12	context switch
12-13	P2 done
13-14	context switch
14-16	P3 (rem 1)
16-17	context switch
17-18	P3 done

- Useful work = $4 + 3 + 5 = 12$ ms
- Context switches = **6**, overhead = **6 ms**
- Total elapsed = **18 ms**
- **CPU efficiency** = $12/18 = 66.7\%$

Formula for RR overhead. With burst B and quantum q , a process needs $\lceil B/q \rceil$ time slices. Total slices = $\sum \lceil B_i/q \rceil$. Context switches $\approx$ total slices (or total slices $- 1$, depending on whether a switch is counted after the last process — **state your convention**).

Check: $\lceil 4/2 \rceil + \lceil 3/2 \rceil + \lceil 5/2 \rceil = 2 + 2 + 3 = 7$ slices, 6 switches between them. ✓

The trade-off question. Smaller quantum → better response time, more context switches, lower efficiency. Larger quantum → fewer switches, RR degenerates toward FCFS. If quantum $\geq$ the largest burst, **RR is exactly FCFS**.

PART 3 — INTER-PROCESS COMMUNICATION

3.1 The two models

Shared memory — a region mapped into both processes' address spaces. They read and write with ordinary memory instructions. Kernel involved only at setup. **Fastest**. Requires you to provide synchronisation.

Message passing — data moves through the kernel via `send()` / `receive()`. Two copies per transfer plus system calls. **Slower**. No explicit synchronisation needed. Works across machines.

3.2 The classification test

Does each transfer require a system call? No → shared memory. Yes → message passing.

A pipe is message passing. This is the single most confused point in the topic. A pipe is a shared buffer — but shared between each process and the **kernel**, not between the two processes' address spaces. Path: `write()` → kernel buffer → `read()`. Two copies.

Textbooks differ (Silberschatz treats pipes as their own section, Tanenbaum as message passing). **In exams, answer message passing.**

3.3 Every mechanism

Mechanism	Model	Direction	Scope	Key property
Shared memory segment	Shared memory	Bidirectional	Any processes	Fastest; <code>shmget/shmat</code> ; needs explicit sync
Memory-mapped file	Shared memory	Bidirectional	Any	<code>mmap</code> ; also used for file I/O
Pipe (anonymous)	Message passing	Unidirectional	Related processes only	Byte stream, no message boundaries, dies with processes
Named pipe / FIFO	Message passing	Uni or bi	Any	Filesystem entry, persists
Message queue	Message passing	Bidirectional	Any	Discrete typed messages; priority retrieval
Socket	Message passing	Bidirectional	Same or different machines	TCP/UDP or Unix domain
Signal	Notification only	One-way	Any (with permission)	Carries only a number — no data
RPC / RMI	Message passing	Request-response	Usually remote	Built over sockets; uses stubs and marshalling
Semaphore	Neither	—	Any	Synchronisation only — no data

3.4 Pipe vs message queue

A pipe is an unstructured **byte stream**: write 10 bytes then 20 bytes and the reader may get all 30 in one `read()`. No boundaries, strict FIFO. A message queue carries **discrete typed messages** that stay intact and can be retrieved by type or priority, out of arrival order.

3.5 Message passing sub-classifications

Naming - Direct — `send(P, msg)`. Link established automatically, exactly two processes per link, bidirectional. - **Indirect** — `send(A, msg)` where A is a **mailbox / port**. Allows many-to-many links. A mailbox can be owned by a process or by the OS.

Synchronisation - Blocking send — sender waits until the message is received. - **Non-blocking send** — sender continues immediately. - **Blocking receive** — receiver waits for a message. - **Non-blocking receive** — returns a message or null. - Blocking send + blocking receive = **rendezvous**.

Buffering — three queue capacities - Zero capacity — no queue; sender must block until the receiver collects. Forces a rendezvous. - **Bounded capacity** — finite queue; sender blocks only when full. - **Unbounded capacity** — sender never blocks.

3.6 IPC trap list

1. **Anonymous pipes are unidirectional.** Two pipes needed for two-way communication. Most-asked IPC question.
2. **Anonymous pipes need a common ancestor** — hence parent-to-child only. Named pipes remove this.
3. **Semaphores and signals transfer no data.** Answers to “which is not a data-transfer mechanism”.
4. **Shared memory is fastest** — no kernel involvement per transfer.
5. **Shared memory is the only model needing explicit synchronisation.**
6. **Pipes have no message boundaries; message queues do.**
7. **A pipe is message passing**, despite involving a buffer.

PART 4 — CONCURRENCY AND SYNCHRONIZATION

4.1 The race condition

Several processes access shared data concurrently; the outcome depends on the interleaving order. The canonical case is a non-atomic increment:

```
x = x + 1
```

compiles to three machine operations:

```
LOAD  R, x
INCR  R
STORE R, x
```

A context switch between any two of these loses an update.

4.2 The critical section problem — three requirements

Requirement	Statement
Mutual exclusion	If one process is in its CS, no other may be in its CS
Progress	If no process is in its CS and some wish to enter, one of those wishing must be allowed in — a process not wanting to enter cannot block others
Bounded waiting	There is a bound on the number of times other processes enter their CS after a process has requested entry and before it is granted

Structure of a solution: entry section → critical section → exit section → remainder section.

Mutual exclusion is the one solutions get right. Progress and bounded waiting are where they fail — that is exactly what questions probe.

4.3 Software solutions and what each violates

Solution	ME	Progress	Bounded wait	Failure mode
Strict alternation (turn variable only)	✓	✗	✓	A process that doesn't want to enter blocks the other; forces alternation
Flag array only (flag[i] = true then wait)	✓	✗	✗	Both set flags → deadlock
Flag set after checking	✗	✓	✓	Both can enter
Peterson's (flag + turn)	✓	✓	✓	Correct for two processes
Dekker's	✓	✓	✓	Correct; predates Peterson's
Bakery algorithm (Lamport)	✓	✓	✓	Works for n processes

Peterson's solution — know the code

```
// shared
boolean flag[2] = {false, false};
int turn;

// process i (other = 1 - i)
flag[i] = true;           // I want in
turn = other;             // but you go first
while (flag[other] && turn == other)
    ;                     // busy wait
critical_section();
flag[i] = false;
```

Why it works: flag[i] expresses intent; turn breaks the tie by *deference* — each process yields the turn to the other, so exactly one of the two while-conditions fails. Satisfies all three requirements. Works only for **two** processes and assumes loads and stores are atomic (fails on architectures that reorder memory operations).

4.4 Hardware support

Instruction	Semantics	Provides
Test-and-Set	Atomically returns the old value and sets the lock to true	ME, progress — not bounded waiting
Compare-and-Swap	Atomically compares and conditionally swaps	ME, progress — not bounded waiting
Swap / Exchange	Atomically exchanges two values	ME, progress
Disable interrupts	Prevents preemption	Works on uniprocessor only; unsafe and non-scalable

Bounded waiting requires extra bookkeeping (a `waiting[]` array) on top of test-and-set. The bare instruction does not provide it — directly asked.

4.5 Semaphores

An integer with two atomic operations:

```
wait(S) / P(S) / down(S):  S--; if (S < 0) block the caller
signal(S) / V(S) / up(S):  S++; if (S <= 0) wake one blocked process
```

Type	Range	Use
Binary (mutex semaphore)	0 or 1	Mutual exclusion
Counting	Unbounded integer	Managing n instances of a resource

Semaphore vs mutex

	Mutex	Semaphore
Value	Locked / unlocked	Integer
Ownership	Only the locking thread may unlock	Any process may signal
Purpose	Mutual exclusion	Signalling and resource counting
Can be used to signal between processes?	No	Yes

Spinlock — a lock that busy-waits instead of blocking. Wastes CPU, but avoids context-switch cost; correct choice when the expected wait is shorter than a context switch.

4.6 Monitors

A high-level construct: shared data plus procedures, with the compiler guaranteeing that **only one process is active inside the monitor at a time**. **Condition variables** support `wait()` and `signal()` within it.

Critical difference from semaphores: a `signal()` on a condition variable with no waiting process is **lost** (no effect). A semaphore `signal()` is remembered in the count. Frequently tested.

Two signalling disciplines: **signal-and-wait** (signaller yields) and **signal-and-continue** (signaller continues).

4.7 Classical synchronization problems

Producer-consumer (bounded buffer)

Three semaphores:

```
mutex = 1    // protects the buffer
empty = n    // count of free slots
full = 0     // count of filled slots
```

```
// Producer          // Consumer
wait(empty);         wait(full);
wait(mutex);         wait(mutex);
    add item to buffer;    remove item from buffer;
signal(mutex);       signal(mutex);
signal(full);        signal(empty);
```

The order matters and is the exam's target. If the producer does `wait(mutex)` before `wait(empty)`, then with a full buffer it holds the mutex while blocked on `empty`, the consumer cannot acquire the mutex to make space → **deadlock**. **Always acquire the counting semaphore before the mutex.**

Readers-writers

Multiple readers may read concurrently; a writer needs exclusive access.

```
mutex = 1    // protects read_count
wrt = 1      // writer lock / first-reader lock
read_count = 0
```

```
// Reader          // Writer
wait(mutex);       wait(wrt);
read_count++;      write;
if (read_count == 1) wait(wrt); signal(wrt);
signal(mutex);
read;
wait(mutex);
read_count--;
if (read_count == 0) signal(wrt);
signal(mutex);
```

First reader locks out writers; last reader releases them. This is the **reader-preference** version and it **starves writers**. Writer-preference and fair variants exist. “Which classical problem can starve one party?” → readers-writers.

Dining philosophers

Five philosophers, five chopsticks, each needs both neighbours' chopsticks.

Naive solution deadlocks — all five pick up their left chopstick simultaneously, all wait forever. This is the standard illustration of **circular wait**.

Fixes: 1. Allow at most **four** philosophers to sit simultaneously. 2. Pick up both chopsticks only if both are available (atomic acquisition). 3. **Asymmetric** — odd-numbered philosophers take left first, even-numbered take right first. Breaks the cycle.

Sleeping barber

One barber, one barber chair, n waiting chairs. Semaphores: `customers = 0`, `barbers = 0`, `mutex = 1`, plus a waiting counter. Illustrates the lost-wakeup problem.

Cigarette smokers

Illustrates that semaphores alone can be insufficient without additional agents/helpers.

4.8 Priority inversion

A high-priority process blocks on a resource held by a low-priority process, while a medium-priority process preempts the low-priority holder — so the highest-priority process waits on the lowest.

Fixes: **priority inheritance** (the holder temporarily inherits the blocked process's priority) or **priority ceiling**.

NUMERICAL TYPE 3 — semaphore final value and blocked count

Final value = $S_0 - (\text{number of waits}) + (\text{number of signals})$ If negative, $|\text{value}| = \text{number of blocked processes}$.

Example 1. $S_0 = 7$; 20 `wait()`, 15 `signal()`. $7 - 20 + 15 = 2$. No process blocked; 2 units free.

Example 2. $S_0 = 3$; 9 `wait()`, 2 `signal()`. $3 - 9 + 2 = -4 \rightarrow 4$ processes blocked.

Example 3 — reverse form. A counting semaphore starts at 10. After some operations its value is -3 and 12 signals were executed. How many waits? $10 - w + 12 = -3 \rightarrow w = 25$.

Example 4 — maximum concurrency. A semaphore initialised to 5 guards a resource. How many processes can be inside the critical section simultaneously? **5**. The initial value is the concurrency limit.

NUMERICAL TYPE 4 — producer-consumer semaphore initialisation

Rule. For a buffer of size n: `mutex = 1`, `empty = n`, `full = 0`.

Example. A bounded buffer holds 10 items. Producers and consumers use semaphores S_1 , S_2 , S_3 . Initial values for mutual exclusion, empty-slot count and filled-slot count? **1, 10, 0**.

Variant — two buffers of different size. A pipeline with buffer A (size 5) and buffer B (size 8) needs `emptyA = 5`, `fullA = 0`, `emptyB = 8`, `fullB = 0`, plus one mutex per buffer.

Variant — producer-only wraparound. If a question gives `full = n` and `empty = 0`, the roles are swapped and the buffer starts full. Read which semaphore guards which condition.

NUMERICAL TYPE 5 — concurrent output range

The signature GATE question type. Two or more processes update a shared variable non-atomically; find the **minimum and maximum** possible final values.

Setup. $x = 0$. Two processes each execute $x = x + 1$ **ten** times. Increment is non-atomic (load, increment, store).

Maximum — no interference; every increment lands. **20**.

Minimum — construct the worst interleaving. P1 loads $x = 0$. P1 is suspended. P2 runs all ten increments $\rightarrow x = 10$. P1 resumes, increments its stale 0 to 1, stores $\rightarrow x = 1$. P1 then runs its remaining nine increments $\rightarrow x = 10$. Now the same trick can be applied once by P2 if it still has work; it doesn't. So minimum = **2**.

General result: with two processes each incrementing n times non-atomically, **maximum = 2n, minimum = 2**.

The minimum is 2, not 1, because each process's *last* store must land, and the two last stores cannot be made to coincide.

Variant — three processes, n increments each. Maximum = $3n$. Minimum = **2** still (two processes can be made to overwrite each other down to 2 regardless of the third's count, provided $n \geq 2$).

Variant — different operations. $x = x + 1$ in P1 (n times) and $x = x + 2$ in P2 (m times). Maximum = $n + 2m$. Minimum = the smaller of the two processes' full contributions overwriting the other's, worked out by interleaving — trace it, don't guess.

Variant — with a semaphore. If both processes wrap the increment in `wait(mutex) / signal(mutex)` with `mutex = 1`, the operation becomes atomic and **minimum = maximum = 2n**. The question is testing whether you notice the semaphore.

NUMERICAL TYPE 6 — which requirement does this solution violate?

Method. Given candidate code, test each requirement by constructing a counterexample.

1. **Mutual exclusion** — can you interleave so both are in the CS? Look for a check-then-set gap.

2. **Progress** — can a process that does *not* want to enter prevent one that does? Look for a `turn` variable that is only updated inside the CS.
3. **Bounded waiting** — can one process enter repeatedly while another waits indefinitely? Look for the absence of any queue or turn-passing.

Worked.

```
// process i
while (turn != i) ;
    critical_section();
turn = 1 - i;
```

- Mutual exclusion: ✓ — only the process whose turn it is proceeds.
- Progress: ✗ — if P0 finishes its CS, sets `turn = 1`, and then never wants to enter again, P1 can enter once and then blocks forever waiting for P0 to hand the turn back. A process not wanting to enter blocks the other.
- Bounded waiting: ✓ — strict alternation bounds the wait at one.

Answer: violates progress. This is strict alternation and it is the most commonly presented broken solution.

NUMERICAL TYPE 7 — maximum processes in the critical section

Rule. With a counting semaphore initialised to k guarding a section, up to k processes can be inside simultaneously.

Composite example. Resource guarded by $S = 3$. Five processes each execute `wait(S); use; signal(S);`. At any instant, how many are using the resource, and how many are blocked at worst? **3 using, 2 blocked.**

Trap. If a question nests two semaphores — `wait(A); wait(B); CS; signal(B); signal(A);` with $A = 4$ and $B = 2$ — the effective concurrency is **$\min(A, B) = 2$** .

PART 5 — DEADLOCK

5.1 The four necessary conditions — all must hold simultaneously

#	Condition	Statement
1	Mutual exclusion	At least one resource is held in non-shareable mode
2	Hold and wait	A process holding at least one resource waits for additional resources held by others
3	No preemption	Resources cannot be forcibly taken; released only voluntarily
4	Circular wait	A set $\{P_0 \dots P_n\}$ exists where each P_i waits for a resource held by P_{i+1} , and P_n waits for P_0

The trap, stated plainly: condition 3 is “**no** preemption”. Any option listing “preemption” as a deadlock condition is the wrong answer. This appeared on your CIL paper.

Relationship: circular wait implies hold and wait, but not conversely. The four are necessary, and for single-instance resources circular wait is also sufficient.

5.2 Deadlock vs starvation vs livelock

	Deadlock	Starvation	Livelock
Processes progress?	No	The starved one, no	They execute but achieve nothing
Cause	Circular wait	Unfair scheduling / priority	Repeated mutual back-off
Resolved by	Breaking the cycle	Ageing	Randomised back-off

5.3 Handling strategies

Strategy	Mechanism	Cost
Prevention	Structurally negate one of the four conditions	Low resource utilisation, low throughput
Avoidance	Require advance knowledge of maximum needs; stay in safe states	Needs max-claim declaration; Banker’s algorithm
Detection & recovery	Allow deadlock; detect with a wait-for graph; recover	Detection overhead; recovery is disruptive
Ignore	Assume it won’t happen	Used by Linux, Windows, UNIX

Prevention — how to attack each condition

Condition	Attack	Consequence
Mutual exclusion	Make resources shareable	Impossible for most resources (printers, write locks)
Hold and wait	Request all resources at once, or release all before requesting more	Low utilisation; possible starvation
No preemption	Preempt resources from waiting processes	Only viable for resources whose state can be saved (CPU, memory) — not printers
Circular wait	Impose a total ordering on resource types; require requests in increasing order	Most practical prevention method

5.4 Resource allocation graph

- **Request edge** $P_i \rightarrow R_j$ · **Assignment edge** $R_j \rightarrow P_i$
- **Single instance per resource type:** a cycle $\implies$ deadlock.
- **Multiple instances:** a cycle is **necessary but not sufficient**. Deadlock may not exist.

This distinction is asked constantly. **No cycle $\rightarrow$ definitely no deadlock**, in both cases.

Wait-for graph — the RAG with resource nodes collapsed out; used for detection with single-instance resources. A cycle means deadlock.

5.5 Safe state

A state is **safe** if a sequence $\langle P_1, P_2, \dots, P_n \rangle$ exists such that each P_i ’s remaining maximum need can be satisfied by the currently available resources plus those held by all P_j with $j < i$.

- **Safe $\rightarrow$ no deadlock.**
- **Unsafe $\rightarrow$ deadlock is possible, not certain.**
- **Deadlock $\subset$ unsafe states.**

5.6 Recovery methods

- **Process termination** — abort all deadlocked processes, or abort one at a time until the cycle breaks. Victim selection by priority, time elapsed, resources held, resources needed, how many need to be aborted, whether the process is interactive or batch.
- **Resource preemption** — select a victim, **rollback** to a safe checkpoint, and guard against **starvation** by including rollback count in the cost factor.

NUMERICAL TYPE 8 — minimum resources, uniform maximum need

Setup. n processes, one resource type, each process may need at most R units.

Worst case: every process holds $R-1$ and waits. Total held = $n(R-1)$. One additional unit lets some process complete.

Deadlock-free guaranteed if $m \geq n(R-1) + 1$ Deadlock possible if $m \leq n(R-1)$

Example 1. 4 processes, each needs at most 3. Minimum units for guaranteed freedom? $4(3-1) + 1 = 9$

Example 2. 5 processes, each needs at most 4. $5(4-1) + 1 = 16$

Example 3. 3 processes each need at most 2 units, and there are 3 units. Deadlock possible? $n(R-1) = 3(1) = 3$. Since $m = 3 \leq 3$, **deadlock is possible**. Need 4 to guarantee freedom.

NUMERICAL TYPE 9 — minimum resources, non-uniform needs

Deadlock-free if $m \geq \sum(R_i - 1) + 1$

Example 1. Three processes with maximum needs 3, 4, 6. $(3-1) + (4-1) + (6-1) + 1 = 2 + 3 + 5 + 1 = 11$

Example 2. Four processes needing 2, 2, 5, 7. $1 + 1 + 4 + 6 + 1 = 13$

Example 3 — reverse. 10 units available; processes need 4, 5 and X . Largest X that still guarantees freedom? $3 + 4 + (X-1) + 1 \leq 10 \rightarrow X + 7 \leq 10 \rightarrow X = 3$

NUMERICAL TYPE 10 — maximum number of processes

Example 1. 15 units; each process needs at most 4. Maximum processes with guaranteed deadlock freedom? $n(4-1) + 1 \leq 15 \rightarrow 3n \leq 14 \rightarrow n \leq 4.67 \rightarrow n = 4$

Example 2. 12 units; each needs at most 3. $2n + 1 \leq 12 \rightarrow n \leq 5.5 \rightarrow n = 5$

Common error: rounding up. You must round **down** — the formula gives an upper bound on n .

NUMERICAL TYPE 11 — Banker's algorithm, safe sequence

Three resource types A, B, C with totals **(10, 5, 7)**.

Process	Allocation	Max	Need = Max - Alloc
P0	0, 1, 0	7, 5, 3	7, 4, 3
P1	2, 0, 0	3, 2, 2	1, 2, 2
P2	3, 0, 2	9, 0, 2	6, 0, 0
P3	2, 1, 1	2, 2, 2	0, 1, 1
P4	0, 0, 2	4, 3, 3	4, 3, 1

Step 1. Total allocated = (7, 2, 5). **Available = (10,5,7) - (7,2,5) = (3, 3, 2)**

Step 2 — safety algorithm. Repeatedly find an unfinished P_i with $\text{Need} \leq \text{Available}$; run it; add its Allocation back to Available.

Iteration	Available	Pick	Test	New Available
1	3,3,2	P1	$1,2,2 \leq 3,3,2 \checkmark$	5,3,2
2	5,3,2	P3	$0,1,1 \leq 5,3,2 \checkmark$	7,4,3
3	7,4,3	P4	$4,3,1 \leq 7,4,3 \checkmark$	7,4,5
4	7,4,5	P0	$7,4,3 \leq 7,4,5 \checkmark$	7,5,5
5	7,5,5	P2	$6,0,0 \leq 7,5,5 \checkmark$	10,5,7

Safe sequence: P1 → P3 → P4 → P0 → P2. State is safe.

Notes that decide marks. - P0 cannot go first — Need (7,4,3) > Available (3,3,2). Candidates who scan top-down and conclude “unsafe” lose the mark. - **Compute the Need matrix first, always.** Errors cluster there, not in the loop. - Multiple safe sequences may exist; any valid one is correct. Here P3 could also go first.

NUMERICAL TYPE 12 — can a specific request be granted?

Three-step procedure. 1. Check **Request ≤ Need** for that process. If not, the process exceeded its declared maximum → **error**. 2. Check **Request ≤ Available**. If not, the process must **wait**. 3. **Pretend to allocate:** Available -= Request, Allocation += Request, Need -= Request. Run the safety algorithm. If the resulting state is **safe**, grant. If **unsafe**, deny and restore.

Worked, continuing the table above. P1 requests **(1, 0, 2)**.

1. Request $(1,0,2) \leq \text{Need } (1,2,2)$ ✓
2. Request $(1,0,2) \leq \text{Available } (3,3,2)$ ✓
3. Pretend: Available = (2,3,0); P1 Allocation = (3,0,2); P1 Need = (0,2,0).

Safety check from Available (2,3,0): - P1 Need $(0,2,0) \leq (2,3,0)$ ✓ $\rightarrow$ Available = (5,3,2) - P3 Need $(0,1,1) \leq (5,3,2)$ ✓ $\rightarrow$ Available = (7,4,3) - P4 Need $(4,3,1) \leq (7,4,3)$ ✓ $\rightarrow$ Available = (7,4,5) - P0 Need $(7,4,3) \leq (7,4,5)$ ✓ $\rightarrow$ Available = (7,5,5) - P2 Need $(6,0,0) \leq (7,5,5)$ ✓ $\rightarrow$ Available = (10,5,7)

Safe sequence exists $\rightarrow$ **request granted**.

Second worked request. Now P4 requests **(3, 3, 0)**. Available is (2,3,0). Request $(3,3,0) \leq (2,3,0)$? **No** — $3 > 2$. P4 must **wait**. No safety check needed.

Third. P0 requests **(0, 2, 0)**. Available (2,3,0). Request $\leq$ Need (7,4,3) ✓, $\leq$ Available ✓. Pretend: Available = (2,1,0), P0 Alloc = (0,3,0), P0 Need = (7,2,3). Safety: P1 Need $(0,2,0) \leq (2,1,0)$? No ($2 > 1$). P3 Need $(0,1,1) \leq (2,1,0)$? No ($1 > 0$). P2 Need $(6,0,0) \leq (2,1,0)$? No. P4 Need $(4,3,1)$? No. P0 Need $(7,2,3)$? No. **No process can proceed $\rightarrow$ unsafe $\rightarrow$ request denied.**

NUMERICAL TYPE 13 — RAG cycle vs deadlock

Rule. Single instance per resource type: cycle $\implies$ deadlock. Multiple instances: cycle is necessary, not sufficient.

Worked. Resource R1 has 2 instances, R2 has 2 instances. P1 holds R1, requests R2. P2 holds R2, requests R1. P3 holds one R1 and one R2, requests nothing.

There is a cycle $P1 \rightarrow R2 \rightarrow P2 \rightarrow R1 \rightarrow P1$. But P3 holds one instance of each and needs nothing more, so P3 completes and releases one R1 and one R2 — satisfying both P1 and P2. **No deadlock despite the cycle.**

PART 6 — CPU SCHEDULING

6.1 Definitions — exact, and worth memorising

Term	Formula
Arrival Time (AT)	When the process enters the ready queue
Burst Time (BT)	CPU time required
Completion Time (CT)	When the process finishes
Turnaround Time (TAT)	$CT - AT$
Waiting Time (WT)	$TAT - BT$
Response Time (RT)	First time on CPU – AT
Throughput	Processes completed per unit time
CPU utilisation	Useful CPU time / total elapsed time

Two subtractions carry every scheduling numerical: build the Gantt chart, read CT, then **TAT = CT – AT** and **WT = TAT – BT**.

Self-check that catches most errors: the Gantt chart must end at (last arrival's idle gaps) + Σ burst times. If there is no idle time, it ends at Σ BT.

6.2 Preemptive vs non-preemptive

Non-preemptive — once scheduled, a process runs to completion or until it blocks. **Preemptive** — the scheduler may remove a running process. Preemption points: quantum expiry, arrival of a higher-priority process, a process becoming ready after I/O.

6.3 All algorithms

Algorithm	Preemptive	Starvation	Optimality	Key weakness
FCFS	No	No	—	Convoy effect ; poor average WT
SJF	No	Yes	Optimal avg WT among non-preemptive	Burst time must be known/predicted
SRTF (preemptive SJF)	Yes	Yes	Optimal average WT overall	High context-switch count
Priority (non-preemptive)	No	Yes	—	Indefinite blocking; fixed by ageing
Priority (preemptive)	Yes	Yes	—	Same
Round Robin	Yes	No	—	Performance highly quantum-dependent
Multilevel Queue	Yes	Possible	—	Rigid; no movement between queues
Multilevel Feedback Queue	Yes	Possible	—	Most general; used in real systems; many tunable parameters
HRRN (Highest Response Ratio Next)	No	No	—	Non-preemptive; ratio = $(W + BT)/BT$
LJF / LRTF	No / Yes	Yes	Worst avg WT	Appears only in exams

Convoy effect — short processes queued behind a long one under FCFS, inflating average waiting time dramatically.

Ageing — gradually increasing the priority of waiting processes. The standard fix for starvation in priority scheduling.

Burst prediction for SJF — exponential averaging: $\tau_{n+1} = \alpha \cdot t_n + (1-\alpha) \cdot \tau_n$. With $\alpha = 0$ only history matters; with $\alpha = 1$ only the last burst matters.

6.4 Quantum behaviour in Round Robin

Quantum	Behaviour
$q \rightarrow \infty$ (or $q \geq \text{max burst}$)	RR becomes exactly FCFS
$q \rightarrow 0$	Ideal fairness (processor sharing) but context-switch overhead dominates; throughput collapses
Rule of thumb	80% of CPU bursts should be shorter than the quantum

NUMERICAL TYPE 14 — the full five-algorithm comparison

One dataset, worked through every algorithm. Learn this and the type is closed.

Process	AT	BT
P1	0	5
P2	1	3
P3	2	8
P4	3	6

$\Sigma BT = 22$, no idle time $\rightarrow$ **every schedule ends at $t = 22$.**

FCFS

	P1		P2		P3		P4		
0		5		8		16		22	

P	CT	TAT	WT
P1	5	5	0
P2	8	7	4
P3	16	14	6
P4	22	19	13

Avg TAT = 11.25 · Avg WT = 5.75

SJF (non-preemptive)

- $t=0$: only P1 present $\rightarrow$ P1 runs 0-5
- $t=5$: P2(3), P3(8), P4(6) available $\rightarrow$ shortest is P2 $\rightarrow$ 5-8
- $t=8$: P3(8), P4(6) $\rightarrow$ P4 $\rightarrow$ 8-14
- $t=14$: P3 $\rightarrow$ 14-22

P	CT	TAT	WT
P1	5	5	0
P2	8	7	4
P3	22	20	12
P4	14	11	5

Avg TAT = 10.75 · Avg WT = 5.25

SRTF (preemptive SJF)

Check for preemption at **every arrival instant**.

- $t=0$: P1 starts, remaining 5
- $t=1$: P2 arrives (3). P1 remaining = 4. $3 < 4 \rightarrow$ **preempt**, P2 runs
- $t=2$: P3 arrives (8). P2 remaining = 2. No preemption
- $t=3$: P4 arrives (6). P2 remaining = 1. No preemption
- $t=4$: P2 completes. Remaining: P1(4), P3(8), P4(6) $\rightarrow$ P1 runs 4-8
- $t=8$: P4(6) < P3(8) $\rightarrow$ P4 runs 8-14
- $t=14$: P3 runs 14-22

	P1		P2		P1		P4		P3		
0		1		4		8		14		22	

P	CT	TAT	WT
P1	8	8	3
P2	4	3	0
P3	22	20	12
P4	14	11	5

Avg TAT = 10.5 · Avg WT = 5.0 — the minimum possible, as SRTF guarantees.

Round Robin, $q = 2$

The rule that decides correctness: a preempted process joins the **back** of the queue, *behind* any process that arrived during its slice.

Time	Running	Queue after
0-2	P1 (rem 3)	P2, P3, P1
2-4	P2 (rem 1)	P3, P1, P4, P2
4-6	P3 (rem 6)	P1, P4, P2, P3
6-8	P1 (rem 1)	P4, P2, P3, P1
8-10	P4 (rem 4)	P2, P3, P1, P4
10-11	P2 done	P3, P1, P4
11-13	P3 (rem 4)	P1, P4, P3
13-14	P1 done	P4, P3
14-16	P4 (rem 2)	P3, P4
16-18	P3 (rem 2)	P4, P3
18-20	P4 done	P3
20-22	P3 done	—

P	CT	TAT	WT	RT
P1	14	14	9	0
P2	11	10	7	1
P3	22	20	12	2
P4	20	17	11	5

Avg TAT = 15.25 · Avg WT = 9.75 · Avg RT = 2.0

Priority (preemptive), lower number = higher priority

Assign priorities P1=3, P2=1, P3=4, P4=2.

- t=0: P1 (only one) runs
- t=1: P2 arrives, priority 1 < 3 → **preempt**. P2 runs 1-4, completes
- t=4: ready are P1(rem 4, pri 3), P3(pri 4), P4(pri 2) → P4 highest → 4-10, completes
- t=10: P1(pri 3) vs P3(pri 4) → P1 → 10-14
- t=14: P3 → 14-22

P	CT	TAT	WT
P1	14	14	9
P2	4	3	0
P3	22	20	12
P4	10	7	1

Avg TAT = 11.0 · Avg WT = 5.5

Summary

Algorithm	Avg TAT	Avg WT
FCFS	11.25	5.75
SJF	10.75	5.25
SRTF	10.5	5.0
Priority (P)	11.0	5.5
RR (q=2)	15.25	9.75

NUMERICAL TYPE 15 — response time

RT = first time on CPU – AT. In FCFS and SJF, RT = WT (a process runs to completion once started). **In RR they differ**, and RR's advantage is precisely its low RT.

From the RR table above: RT = 0, 1, 2, 5 → **avg RT 2.0** versus avg WT 9.75. That gap is the entire justification for RR.

Worked — all arrive at t = 0, bursts 5, 3, 8, q = 2. First CPU times: P1 at 0, P2 at 2, P3 at 4. **RT = 0, 2, 4 → avg 2.0.**

General: all processes arriving at t=0 with quantum q, the k-th process in queue order has RT = (k–1)·q.

NUMERICAL TYPE 16 — RR with context switch overhead

Worked. Three processes at t = 0, bursts 4, 3, 5. q = 2, context switch = 1 ms.

Time	Activity
0-2	P1 (rem 2)
2-3	CS
3-5	P2 (rem 1)
5-6	CS
6-8	P3 (rem 3)
8-9	CS
9-11	P1 done (CT 11)
11-12	CS
12-13	P2 done (CT 13)
13-14	CS
14-16	P3 (rem 1)
16-17	CS
17-18	P3 done (CT 18)

- Useful work = 12 ms · Switches = 6 · Overhead = 6 ms · Total = 18 ms
- **CPU efficiency = 12/18 = 66.7%**
- TAT (AT=0) = 11, 13, 18 → avg **14** · WT = TAT – BT = 7, 10, 13 → avg **10**

Slice count formula: $\Sigma[B_i/q] = [4/2] + [3/2] + [5/2] = 2 + 2 + 3 = 7$ **slices**, hence 6 switches between them.

NUMERICAL TYPE 17 — number of context switches

Non-preemptive algorithms: switches = $n - 1$ (one per handover), where n is the number of processes.

Round Robin: switches = $\Sigma[B_i/q] - 1$, provided no process's remaining time is an exact multiple concern. Watch for the convention on the final switch.

SRTF: count preemption points plus completions. From the SRTF trace above: P1→P2 at $t=1$, P2→P1 at $t=4$, P1→P4 at $t=8$, P4→P3 at $t=14$ → **4 switches**.

NUMERICAL TYPE 18 — throughput and CPU utilisation

Throughput = processes completed / total time CPU utilisation = useful CPU time / total elapsed time

Worked. From the RR-with-overhead example: 3 processes in 18 ms → throughput = $3/18 = 0.167$ **processes/ms**. Utilisation = **66.7%**.

Worked with idle time. Processes arrive at $t = 0, 5, 12$ with bursts 3, 4, 2 under FCFS. - 0-3 P1; 3-5 **idle**; 5-9 P2; 9-12 **idle**; 12-14 P3 - Total elapsed = 14, useful = 9 → **utilisation 64.3%**, throughput = $3/14 = 0.214$ /**ms**

NUMERICAL TYPE 19 — quantum size effects

Worked. Processes at $t=0$, bursts 6, 3, 8.

Quantum	Behaviour	Avg WT
$q = 1$	Near processor-sharing; maximum switches	Highest
$q = 3$	Balanced	Moderate
$q = 8$	$q \geq \max \text{burst} \rightarrow$ degenerates to FCFS	Same as FCFS

Degeneration threshold question. "For what minimum quantum does RR behave identically to FCFS?" → **$q \geq$ the largest burst time**. Here **$q \geq 8$** .

NUMERICAL TYPE 20 — which algorithm minimises average WT

Rule. For any fixed set of processes: - With all arrivals at $t = 0$, **SJF is optimal** (and identical to SRTF). - With staggered arrivals, **SRTF is optimal**. - **LJF/LRTF give the worst** average WT. - RR is never optimal for WT but is best for RT.

Do not compute all five. Identify the arrival pattern, name the optimal algorithm, and compute only that one if a value is required.

PART 7 — MEMORY MANAGEMENT

7.1 Address binding and translation

- **Compile time** — absolute addresses generated; program must load at a fixed location.
- **Load time** — relocatable code; binding fixed at load.
- **Execution time** — binding deferred to run time; requires hardware (MMU). Needed for swapping and virtual memory.

Logical (virtual) address — generated by the CPU. **Physical address** — seen by the memory unit. **MMU** translates at run time.

Simple relocation scheme: physical = logical + **base (relocation) register**, after checking logical < **limit register**. A violation traps to the OS as an addressing error.

7.2 Swapping

A process is moved wholly to backing store and back. **Transfer time dominates** and is proportional to the amount of memory swapped. A process with pending I/O cannot be swapped out safely unless I/O goes through kernel buffers.

7.3 Contiguous allocation

	Fixed (static) partitioning	Variable (dynamic) partitioning
Partition creation	At boot	Per process, exactly sized
Primary waste	Internal fragmentation	External fragmentation
Degree of multiprogramming	Fixed by partition count	Variable
Needs compaction	No	Yes

Internal fragmentation — unused space *inside* an allocated block, because the block is larger than the request. Occurs in fixed partitioning and in **paging** (last page only).

External fragmentation — total free memory is sufficient but not contiguous. Occurs in variable partitioning and **segmentation**. Remedy: **compaction** (requires execution-time binding) or paging.

The 50% rule: with first fit, statistically about one third of memory may be unusable to external fragmentation (N allocated blocks → 0.5N lost blocks).

NUMERICAL TYPE 21 — allocation strategies

Partitions in order: 100 KB, 500 KB, 200 KB, 300 KB, 600 KB **Process requests in order:** 212 KB, 417 KB, 112 KB, 426 KB

First fit — first partition large enough, scanning from the start.

Request	Partition	Remainder
212	500	288
417	600	183
112	288 (remainder of 500)	176
426	no fit	waits

Best fit — smallest partition that fits.

Request	Partition	Remainder
212	300	88
417	500	83
112	200	88
426	600	174

All four accommodated.

Worst fit — largest available partition.

Request	Partition	Remainder
212	600	388
417	500	83
112	388 (remainder of 600)	276
426	no fit	waits

Next fit — like first fit but resumes scanning from where the last allocation ended, rather than restarting.

Summary: best fit is the only strategy that accommodates all four here — that is the point of the dataset. But best fit is **slowest** (must scan the whole list) and leaves many tiny unusable holes. First fit is fastest. Worst fit aims to leave large usable remainders but performs poorly in practice.

NUMERICAL TYPE 22 — internal fragmentation amount

Fixed partitioning. Partitions 4 KB, 8 KB, 8 KB, 16 KB. Processes of 3 KB, 6 KB, 7 KB, 12 KB allocated in order using first fit.

Process	Partition	Internal fragmentation
3 KB	4 KB	1 KB
6 KB	8 KB	2 KB
7 KB	8 KB	1 KB
12 KB	16 KB	4 KB

Total internal fragmentation = 8 KB

Paging. A process of 21 KB with a 4 KB page size needs $\lceil 21/4 \rceil = 6$ pages = 24 KB allocated. **Internal fragmentation = 3 KB**, all in the last page.

Average internal fragmentation in paging = half a page per process. Smaller pages reduce it but enlarge the page table — that is the page-size trade-off question.

NUMERICAL TYPE 23 — external fragmentation

Worked. Memory of 1 MB. Allocated: 200 KB at 0, then a 100 KB hole, then 300 KB, then a 150 KB hole, then 200 KB, then a 50 KB hole.

- Total free = 100 + 150 + 50 = **300 KB**
- A request for **250 KB** cannot be satisfied despite 300 KB free → **external fragmentation**
- After **compaction**, all 300 KB is contiguous and the request succeeds.

Question form: “Total free memory is X, largest request satisfiable is Y” → Y is the largest single hole, and X – Y quantifies the fragmentation cost.

7.4 Paging

- **Page** = fixed-size block of logical memory. **Frame** = same-size block of physical memory. Page size = frame size, always a **power of 2**.
- **Page table** — one per process, indexed by page number, yielding the frame number.
- **PTE contents:** frame number, **valid/invalid bit**, protection bits (read/write/execute), **dirty (modified) bit**, **reference bit**, caching-disabled bit.
- Paging eliminates external fragmentation entirely; introduces bounded internal fragmentation.
- **Page table base register (PTBR)** points to the current process’s page table; changed on context switch.

NUMERICAL TYPE 24 — address bit splitting

Offset bits = $\log_2(\text{page size})$ Page number bits = logical address bits – offset bits Frame number bits = $\log_2(\text{physical memory} / \text{frame size})$

Example 1. 32-bit logical address, 4 KB pages. Offset = 12, page number = **20 bits**, number of pages = $2^{20} = 1\text{M}$.

Example 2. 16-bit logical address, 1 KB pages, 64 KB physical memory. Offset = 10 · page number = 6 bits (64 pages) · frames = 64 KB/1 KB = 64 → frame number = **6 bits** · physical address = 16 bits.

Example 3 — asymmetric. 32-bit logical, 4 GB physical, 8 KB pages. Offset = 13 · pages = 2^{19} · frames = $2^{32}/2^{13} = 2^{19} \rightarrow$ **19 bits** each.

Example 4 — physical larger than logical. 32-bit logical, 64 GB (2^{36}) physical, 4 KB pages. Offset 12 · pages 2^{20} · frames $2^{36}/2^{12} = 2^{24} \rightarrow$ frame number **24 bits**. Physical address is 36 bits, wider than the logical address. Perfectly legal and commonly asked.

NUMERICAL TYPE 25 — frames and frame bits

Example. 1 GB physical memory, 4 KB frames. Frames = $2^{30}/2^{12} = 2^{18} =$ **262,144**. Frame number = **18 bits**.

Example. 512 MB physical, 8 KB frames. $2^{29}/2^{13} = 2^{16} = 65,536$ frames → **16 bits**.

7.5 Segmentation

Memory divided into **variable-length logical units** matching the program’s structure (code segment, data segment, stack segment, individual arrays or functions).

Logical address = {segment number, offset}

Segment table entry: **base** (starting physical address) and **limit** (segment length).

Translation: if offset < limit, physical = base + offset. Otherwise **trap: addressing error**.

	Paging	Segmentation
Block size	Fixed	Variable
Divided by	Hardware/OS	Programmer's logical view
Fragmentation	Internal	External
Address	One-dimensional	Two-dimensional (segment, offset)
Sharing/protection	Per page — awkward	Per segment — natural
Table entry	Frame number	Base + limit

NUMERICAL TYPE 26 — segmentation translation

Segment table:

Segment	Base	Limit
0	1200	500
1	3400	200
2	4300	700
3	2100	100

Translate each logical address:

Logical address	Check	Physical address
(0, 320)	$320 < 500$ ✓	$1200 + 320 = \mathbf{1520}$
(1, 250)	$250 < 200$ ✗	Trap — addressing error
(2, 600)	$600 < 700$ ✓	$4300 + 600 = \mathbf{4900}$
(3, 99)	$99 < 100$ ✓	$2100 + 99 = \mathbf{2199}$
(3, 100)	$100 < 100$ ✗	Trap — offset must be strictly less than limit

The boundary case (3, 100) is the trap. Valid offsets are 0 to limit–1.

NUMERICAL TYPE 27 — segmentation with paging

Each segment is itself paged. **Logical address = (segment number | page number | offset)**

Worked. 32-bit logical address. 16 segments. Page size 4 KB. Compute the bit split.

- Segments = $16 = 2^4 \rightarrow$ **segment number = 4 bits**
- Page size 4 KB = $2^{12} \rightarrow$ **offset = 12 bits**
- **Page number = $32 - 4 - 12 = 16$ bits** $\rightarrow$ each segment may hold up to 2^{16} pages = 256 MB

Split: 4 | 16 | 12

Translation steps: segment number indexes the **segment table** $\rightarrow$ yields the base address of that segment's **page table** $\rightarrow$ page number indexes it $\rightarrow$ yields the frame number $\rightarrow$ frame number concatenated with offset gives the physical address. **Two table lookups plus the data access = three memory accesses without a TLB.**

Worked variant. 24-bit logical address, 8 segments, 2 KB pages. Segment = 3 bits $\cdot$ offset = 11 bits $\cdot$ page number = $24 - 3 - 11 = \mathbf{10}$ bits. Maximum segment size = $2^{10} \times 2$ KB = **2 MB**.

PART 8 — VIRTUAL MEMORY

8.1 The idea

Execute processes not entirely in memory. **Demand paging** loads a page only when referenced. Benefits: logical address space larger than physical memory, higher degree of multiprogramming, less I/O to load a process, faster process creation via **copy-on-write**.

Valid/invalid bit drives it: an invalid bit on a referenced page causes a **page fault** trap.

8.2 Page fault handling — the full sequence

1. Reference a page; MMU finds the valid/invalid bit clear
2. **Trap to the OS**
3. OS checks whether the reference is legal (in the process's address space) — if illegal, **terminate**; if legal but not resident, continue
4. Find a **free frame**; if none, run **page replacement** to select a victim
5. If the victim is **dirty**, write it to disk first
6. **Schedule a disk read** for the required page
7. Process blocks; another process runs
8. On I/O completion, update the page table and set the valid bit
9. **Restart the instruction** that faulted

A **page fault is not an error**. It is a normal, expected trap. Options calling it an error or exception-to-be-terminated are distractors.

Pure demand paging — start a process with *no* pages resident; every first reference faults.

8.3 Copy-on-write

`fork()` initially shares all pages between parent and child, marked copy-on-write. A page is duplicated only when one of them writes to it. Makes process creation cheap. `vfork()` skips this entirely and shares the address space.

8.4 TLB

A small, fast associative cache of recent page-table entries. On a **TLB hit**, translation costs only the TLB lookup. On a **miss**, the page table must be walked.

TLB and context switches: the TLB must be **flushed** on a context switch unless entries carry **ASIDs** (address-space identifiers). Asked directly.

NUMERICAL TYPE 28 — single-level page table size

Page table size = number of pages × PTE size **Number of pages = logical address space / page size**

Example 1. 32-bit virtual address, 4 KB page, 4-byte PTE. Pages = $2^{32}/2^{12} = 2^{20} \rightarrow \text{size} = 2^{20} \times 4 = \mathbf{4\ MB}$

Example 2. 36-bit virtual address, 8 KB page, 4-byte PTE. Offset 13 $\rightarrow$ pages = $2^{23} \rightarrow \text{size} = 2^{23} \times 4 = 2^{25} = \mathbf{32\ MB}$

Example 3. 48-bit virtual address, 8 KB page, 8-byte PTE. Pages = $2^{35} \rightarrow \text{size} = 2^{35} \times 8 = 2^{38} = \mathbf{256\ GB}$. This absurd figure is the entire motivation for multilevel paging.

Example 4 — deriving PTE size. 32-bit logical, 16 GB physical, 4 KB pages. What is the minimum PTE size? Frames = $2^{34}/2^{12} = 2^{22} \rightarrow$ frame number needs 22 bits. Plus valid, dirty, reference and protection bits, say 4 $\rightarrow$ 26 bits $\rightarrow$ round to **4 bytes**. Page table size = $2^{20} \times 4 = \mathbf{4\ MB}$.

NUMERICAL TYPE 29 — multilevel paging bit split

Governing rule: each page table must fit in exactly one page. Entries per page table = page size / PTE size

Example 1. 32-bit address, 4 KB page, 4-byte PTE. Two-level design. - Offset = 12 - Entries per table = $4096/4 = 1024 = 2^{10} \rightarrow$ **inner index = 10 bits** - Outer index = $32 - 12 - 10 = \mathbf{10\ bits}$ - **Split: 10 | 10 | 12**

Example 2. 32-bit address, 8 KB page, 4-byte PTE. - Offset = 13 - Entries = $8192/4 = 2048 = 2^{11} \rightarrow$ inner = 11 bits - Outer = $32 - 13 - 11 = \mathbf{8\ bits}$ - **Split: 8 | 11 | 13**

Example 3 — three-level. 48-bit address, 4 KB page, 8-byte PTE. - Offset = 12 - Entries = $4096/8 = 512 = 2^9 \rightarrow$ 9 bits per level - Page number bits = $48 - 12 = 36 \rightarrow 36/9 = \mathbf{4\ levels}$ - **Split: 9 | 9 | 9 | 12**

NUMERICAL TYPE 30 — number of levels required

Levels = [(page number bits) / log₂(entries per page table)]

Example 1. 32-bit address, 1 KB page, 4-byte PTE. - Offset = 10 $\rightarrow$ page number = 22 bits - Entries per table = $1024/4 = 256 = 2^8 \rightarrow$ 8 bits per level - Levels = $\lceil 22/8 \rceil = \mathbf{3}$ - Split: **6 | 8 | 8 | 10** (top level uses only the leftover 6 bits)

Example 2. 40-bit address, 4 KB page, 4-byte PTE. - Offset 12 $\rightarrow$ page number 28 bits · 10 bits per level - Levels = $\lceil 28/10 \rceil = \mathbf{3} \rightarrow$ split **8 | 10 | 10 | 12**

Example 3 — how many pages does the outer table occupy? 32-bit, 4 KB page, 4-byte PTE, **single** level: 4 MB table $\div$ 4 KB page

= **1024 pages**. Two-level: the outer table has 2^{10} entries $\times 4 \text{ B} = 4 \text{ KB} = \text{exactly one page}$. That is the design goal.

NUMERICAL TYPE 31 — inverted page table size

An **inverted page table** has one entry per **frame**, not per page — one table for the entire system.

Example. 4 GB physical memory, 4 KB frames, each entry storing a PID (say 4 bytes) and a page number (4 bytes). Frames = $2^{32}/2^{12} = 2^{20} \rightarrow \text{size} = 2^{20} \times 8 = \mathbf{8 \text{ MB total for the whole system}}$, versus 4 MB *per process* with conventional paging.

Trade-off: enormous space saving, but lookup requires searching by (PID, page) rather than indexing — so it is hashed. Also makes **shared memory hard to represent**, since one frame can map to only one entry.

NUMERICAL TYPE 32 — TLB EAT, hierarchical access

TLB is searched first; on a miss, the page table is consulted, then the data.

$$\text{EAT} = h(t_{\text{TLB}} + t_{\text{mem}}) + (1 - h)(t_{\text{TLB}} + 2 \cdot t_{\text{mem}})$$

Example 1. TLB 20 ns, memory 100 ns, hit ratio 80%. - Hit: $0.8 \times (20 + 100) = 96$ - Miss: $0.2 \times (20 + 100 + 100) = 44$ - **EAT = 140 ns**

Example 2. TLB 10 ns, memory 80 ns, hit ratio 90%. - $0.9(90) + 0.1(170) = 81 + 17 = \mathbf{98 \text{ ns}}$

Example 3 — reverse form. Memory 100 ns, TLB 20 ns. What hit ratio gives an EAT of 130 ns? $130 = h(120) + (1-h)(220) \rightarrow 130 = 220 - 100h \rightarrow h = \mathbf{0.9}$

NUMERICAL TYPE 33 — TLB EAT, parallel access

If the TLB is searched **in parallel with** memory (or the TLB time is stated as negligible):

$$\text{EAT} = h \cdot t_{\text{mem}} + (1 - h) \cdot 2 \cdot t_{\text{mem}}$$

Example. Memory 100 ns, hit ratio 80%. $0.8(100) + 0.2(200) = \mathbf{120 \text{ ns}}$

Same inputs, 140 ns hierarchical vs 120 ns parallel. The wording decides. Look for “TLB is searched first” (hierarchical) versus “TLB lookup time is negligible” or “searched simultaneously” (parallel).

NUMERICAL TYPE 34 — TLB EAT with a multilevel page table

On a TLB miss, **each level costs one memory access**, plus one for the data.

$$\text{EAT} = h(t_{\text{TLB}} + t_{\text{mem}}) + (1 - h)(t_{\text{TLB}} + (L+1) \cdot t_{\text{mem}}) \text{ where } L = \text{number of levels}$$

Example. TLB 20 ns, memory 100 ns, two-level page table, hit ratio 85%. - Hit: $0.85(20 + 100) = 102$ - Miss: $0.15(20 + 200 + 100) = 0.15 \times 320 = 48$ - **EAT = 150 ns**

Example — three-level, hit ratio 95%, TLB 10 ns, memory 100 ns. - Hit: $0.95(110) = 104.5$ - Miss: $0.05(10 + 300 + 100) = 0.05 \times 410 = 20.5$ - **EAT = 125 ns**

NUMERICAL TYPE 35 — page fault EAT

$$\text{EAT} = (1 - p) \cdot t_{\text{mem}} + p \cdot t_{\text{fault}}$$

Example 1. Memory 100 ns, page fault service 8 ms, fault rate 0.001. $8 \text{ ms} = 8,000,000 \text{ ns}$ $\text{EAT} = 0.999(100) + 0.001(8,000,000) = 99.9 + 8,000 = \mathbf{8,099.9 \text{ ns} \approx 8.1 \mu\text{s}}$

Access time degraded **81×** by a 0.1% fault rate. This is the quantitative answer to “why is thrashing catastrophic”.

Example 2. Memory 200 ns, fault service 10 ms, fault rate 0.0005. $0.9995(200) + 0.0005(10,000,000) = 199.9 + 5,000 = \mathbf{5,199.9 \text{ ns}}$

Example 3 — slowdown factor. For example 1: $8099.9/100 = \mathbf{81\times \text{slowdown}}$.

NUMERICAL TYPE 36 — reverse form: required fault rate

Example 1. Memory 100 ns, fault service 8 ms. What fault rate keeps degradation below **10%**?

Target $\text{EAT} \leq 110 \text{ ns}$. $110 = (1-p)100 + p(8,000,000)$ $110 = 100 + 7,999,900p$ $p = 10/7,999,900 = \mathbf{1.25 \times 10^{-6} \rightarrow \text{roughly one fault per 800,000 accesses}}$

Example 2. Memory 200 ns, fault service 8 ms, degradation limit 20%. Target 240 ns. $240 = 200 + p(8,000,000 - 200) \rightarrow 40 = 7,999,800p \rightarrow p = \mathbf{5 \times 10^{-6}}$

Method: set target = $t_{\text{mem}} \times (1 + \text{allowed degradation})$, then solve. The t_{mem} term inside the bracket is usually negligible but keep it for exactness.

NUMERICAL TYPE 37 — dirty page EAT

If a fraction d of replaced pages are dirty, they must be written out before the new page is read — doubling the disk cost for those faults.

$$\text{Average fault cost} = (1 - d) \cdot t_{\text{fault}} + d \cdot 2 \cdot t_{\text{fault}} = t_{\text{fault}}(1 + d) \quad \text{EAT} = (1 - p) \cdot t_{\text{mem}} + p \cdot t_{\text{fault}}(1 + d)$$

Example. Memory 100 ns, fault service 10 ms, $p = 0.002$, 40% of victims dirty. - Average fault cost = $10 \text{ ms} \times 1.4 = 14 \text{ ms} = 14,000,000 \text{ ns}$ - $\text{EAT} = 0.998(100) + 0.002(14,000,000) = 99.8 + 28,000 = \mathbf{28,099.8 \text{ ns}}$

8.5 Page replacement algorithms

Algorithm	Belady's anomaly	Implementable	Mechanism
FIFO	Yes	Yes	Replace the oldest loaded page
Optimal (OPT/MIN)	No	No	Replace the page needed furthest in the future
LRU	No	Yes, expensively	Replace the least recently used page
Second chance / Clock	No	Yes	FIFO + reference bit; give a second chance if the bit is set
Enhanced second chance	No	Yes	Uses (reference, dirty) pairs; four classes
LFU	Possible	Yes	Least frequently used
MFU	Possible	Yes	Most frequently used (argues the low-count page just arrived)

Belady's anomaly — more frames producing **more** page faults. Occurs in **FIFO**. **Never** in LRU or Optimal, because those are **stack algorithms**: the set of pages in k frames is always a subset of the set in $k+1$ frames.

LRU implementation — counters (timestamp per PTE, search for the minimum) or a **stack** of page numbers (move the referenced page to the top; the bottom is LRU). Both need hardware support to be practical.

NUMERICAL TYPE 38 — page fault counting, four algorithms

Canonical reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

FIFO, 3 frames

Ref	Frames (oldest → newest)	Result
1	1	F
2	1, 2	F
3	1, 2, 3	F
4	2, 3, 4	F evict 1
1	3, 4, 1	F evict 2
2	4, 1, 2	F evict 3
5	1, 2, 5	F evict 4
1	1, 2, 5	hit
2	1, 2, 5	hit
3	2, 5, 3	F evict 1
4	5, 3, 4	F evict 2
5	5, 3, 4	hit

9 faults, 3 hits. Hit ratio = $3/12 = 25\%$.

FIFO, 4 frames

Ref	Frames	Result
1	1	F
2	1,2	F
3	1,2,3	F
4	1,2,3,4	F
1	1,2,3,4	hit
2	1,2,3,4	hit
5	2,3,4,5	F evict 1
1	3,4,5,1	F evict 2
2	4,5,1,2	F evict 3
3	5,1,2,3	F evict 4
4	1,2,3,4	F evict 5
5	2,3,4,5	F evict 1

10 faults. More frames, more faults → **Belady's anomaly**. This exact 9-then-10 pair is the standard exam demonstration.

LRU, 3 frames

Ref	Frames (LRU → MRU)	Result
1	1	F
2	1, 2	F
3	1, 2, 3	F
4	2, 3, 4	F evict 1 (LRU)
1	3, 4, 1	F evict 2
2	4, 1, 2	F evict 3
5	1, 2, 5	F evict 4
1	2, 5, 1	hit
2	5, 1, 2	hit
3	1, 2, 3	F evict 5
4	2, 3, 4	F evict 1
5	3, 4, 5	F evict 2

10 faults. Note **LRU is worse than FIFO here**. LRU is better *on average*, never guaranteed better on a specific string. Questions exploit this assumption.

Optimal, 3 frames

Ref	Frames	Result	Reasoning
1	1	F	
2	1,2	F	
3	1,2,3	F	
4	1,2,4	F	next use: 1→pos5, 2→pos6, 3→pos10 → evict 3
1	1,2,4	hit	
2	1,2,4	hit	
5	1,2,5	F	next use: 1→8, 2→9, 4→11 → evict 4
1	1,2,5	hit	
2	1,2,5	hit	
3	2,3,5	F	1 never used again → evict 1
4	3,4,5	F	2 never used again → evict 2
5	3,4,5	hit	

7 faults — the theoretical minimum for 3 frames on this string.

Clock / second chance, 3 frames

Maintain a circular list with a reference bit. On a fault, advance the pointer; if the current page's reference bit is 1, clear it and move on; if 0, replace it. A referenced page gets one reprieve.

On short strings Clock usually matches FIFO or falls between FIFO and LRU. **Its purpose is to approximate LRU at FIFO's cost**, which is the conceptual point being tested.

Summary

Algorithm	Frames	Faults	Hit ratio
Optimal	3	7	41.7%
FIFO	3	9	25%
LRU	3	10	16.7%
FIFO	4	10	16.7%

Execution method: draw one column per reference, write frame contents down each column, mark F for faults, **count the F marks at the end**. Do not maintain a running total — that is where slips happen.

NUMERICAL TYPE 39 — demonstrating Belady's anomaly

Standard string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 with FIFO → 9 faults at 3 frames, 10 at 4 frames.

Minimal string. 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 is the textbook case. Shorter anomalous strings exist but are not asked.

Question forms: - "Which algorithm can exhibit Belady's anomaly?" → **FIFO** (and LFU/MFU in principle) - "Why can LRU never exhibit it?" → **LRU is a stack algorithm**; pages in k frames $\subseteq$ pages in k+1 frames - "Increasing frames always reduces faults — true or false?" → **False**, for FIFO

NUMERICAL TYPE 40 — hit ratio and fault rate

Hit ratio = hits / total references · Fault rate = faults / total references · The two sum to 1

Worked. FIFO 3 frames above: 9 faults, 3 hits, 12 references → fault rate = $9/12 = 75\%$, hit ratio **25%**.

Combined with EAT. Memory 100 ns, fault service 5 ms, fault rate 75% (a thrashing scenario): $EAT = 0.25(100) + 0.75(5,000,000) = 25 + 3,750,000 = 3.75 \text{ ms per access}$. Illustrates why this fault rate is unusable.

NUMERICAL TYPE 41 — working set and thrashing threshold

Working set $W(t, \Delta)$ = the set of distinct pages referenced in the most recent Δ references.

Worked. String: 2, 6, 1, 5, 7, 7, 7, 5, 1, 6, 2, 3, 4, 1, 2, 3. With $\Delta = 5$, the working set at each point is the distinct pages in the last 5 references.

- After reference 5 (...2,6,1,5,7): $W = \{1, 2, 5, 6, 7\} \rightarrow \text{size } 5$
- After reference 7 (...5,7,7,7 with 1 before): window = 1,5,7,7,7 → $W = \{1, 5, 7\} \rightarrow \text{size } 3$
- After reference 11 (7,5,1,6,2): $W = \{1, 2, 5, 6, 7\} \rightarrow \text{size } 5$

Thrashing condition. $> \text{If } \sum WSS_i \text{ (total working set demand)} > \text{total available frames} \rightarrow \text{thrashing}$

Worked. Three processes with working set sizes 20, 35 and 30 pages. Total demand = 85 frames. If the system has 100 frames, no thrashing. If it has 70, **thrashing occurs** and the degree of multiprogramming must be reduced.

Page fault frequency (PFF) scheme — set upper and lower bounds on the acceptable fault rate. Above the upper bound, allocate more frames; below the lower bound, take frames away. If no frames can be freed, **suspend a process**.

8.6 Thrashing — theory

Definition: the page fault rate is so high that the system spends more time paging than executing.

Mechanism: the degree of multiprogramming rises → each process gets fewer frames → fault rate rises → processes block on I/O → CPU utilisation falls → the long-term scheduler *adds more processes* to raise utilisation → the situation worsens catastrophically.

Signature: CPU utilisation collapses while disk utilisation saturates.

Remedies: reduce the degree of multiprogramming (suspend or swap out processes), use local rather than global replacement, apply the working set model, apply the PFF scheme, or add physical memory. **Adding processes is the opposite of the fix** — that is the trap.

8.7 Frame allocation

- **Equal allocation** — m/n frames each.
- **Proportional allocation** — frames $\propto$ process size (or priority).
- **Global replacement** — a process may take a frame from another. Higher throughput, unpredictable per-process behaviour, and **a process cannot control its own fault rate**.
- **Local replacement** — a process replaces only its own frames. Predictable but may under-use memory.

Minimum frames per process is dictated by the instruction set: an instruction with multiple memory operands must have all its pages resident simultaneously or it can never complete.

PART 9 — FILE SYSTEMS

9.1 File concept

Attributes: name, identifier, type, location, size, protection, timestamps (creation, last access, last modification), owner.

Operations: create, write, read, reposition (seek), delete, truncate, open, close.

Open file table — the OS keeps a **system-wide** table (one entry per open file, with a reference count) and a **per-process** table (with the current file pointer and access rights). The **file descriptor** indexes the per-process table.

9.2 Access methods

Method	Mechanism	Suited to
Sequential	Read/write in order; rewind supported	Tapes, logs, streams
Direct / relative	Read/write block n directly	Databases
Indexed	Index file maps keys to block numbers	Large keyed records

9.3 Directory structures

Structure	Property	Limitation
Single level	One directory for all files	Name collisions; no grouping
Two level	One directory per user	No grouping within a user; sharing awkward
Tree	Arbitrary nesting; unique path names	No sharing of a file between directories
Acyclic graph	Sharing via links	Must prevent cycles; deletion needs reference counts
General graph	Cycles allowed	Needs garbage collection ; traversal may loop

Hard link — a second directory entry pointing to the same inode. Reference count in the inode; deleting one entry does not free the file until the count reaches zero. **Symbolic (soft) link** — a file containing a path. Can dangle if the target is deleted. Can cross filesystems; hard links cannot.

9.4 Allocation methods — the core comparison

Method	Random access	External fragmentation	Space overhead	File growth
Contiguous	Excellent — $O(1)$	Yes	Minimal (start + length)	Difficult
Linked	Poor — must traverse	No	One pointer per block	Easy
Indexed	Good — via index block	No	One index block per file	Easy
FAT	Moderate (FAT cached in memory)	No	FAT table for whole disk	Easy

Linked allocation variants: simple linked list; **FAT** (File Allocation Table — pointers pulled out into a single table at the start of the volume, which can be cached, greatly improving random access).

Indexed allocation variants for large files: - **Linked scheme** — last index-block entry points to another index block - **Multilevel index** — an index block of index blocks - **Combined scheme (UNIX inode)** — direct pointers plus single, double and triple indirect

NUMERICAL TYPE 42a — disk accesses per allocation method

Contiguous: 1 access (compute the address arithmetically) **Linked:** n accesses to reach block n (must traverse) **Indexed, single level:** 2 accesses (index block + data block) **Indexed, two level:** 3 accesses **FAT (table cached in memory):** 1 access (table lookups are free)

Worked 1. A file uses **linked** allocation. Disk accesses to read the **10th** block? Traverse blocks 1 through 9, then read block 10 → **10 accesses**.

Worked 2. Same file under **contiguous** allocation → **1 access**.

Worked 3. Same file under **single-level indexed** → **2 accesses** (read the index block, then block 10).

Worked 4 — insertion cost. To insert a block in the **middle** of a linked-allocation file, you must reach the preceding block (n accesses), write the new block, and update one pointer. Under contiguous allocation, insertion requires **shifting all subsequent blocks** — the standard “which allocation method makes insertion cheapest” question answers **linked**.

NUMERICAL TYPE 42b — inode maximum file size

Worked 1. Inode with **12 direct** pointers, 1 single indirect, 1 double indirect, 1 triple indirect. Block size **4 KB**, pointer size **4 bytes**.
Pointers per block = $4096/4 = 1024$

Level	Blocks addressed	Bytes
Direct	12	$12 \times 4 \text{ KB} = \mathbf{48 \text{ KB}}$
Single indirect	1024	$1024 \times 4 \text{ KB} = \mathbf{4 \text{ MB}}$
Double indirect	$1024^2 = 1,048,576$	$\mathbf{4 \text{ GB}}$
Triple indirect	1024^3	$\mathbf{4 \text{ TB}}$

Maximum file size = 4 TB + 4 GB + 4 MB + 48 KB ≈ 4 TB

Worked 2. Inode with **10 direct**, 1 single indirect, 1 double indirect. Block 1 KB, pointer 4 bytes. Pointers per block = $1024/4 = \mathbf{256}$ - Direct: $10 \times 1 \text{ KB} = 10 \text{ KB}$ - Single: $256 \times 1 \text{ KB} = 256 \text{ KB}$ - Double: $256^2 \times 1 \text{ KB} = 65,536 \text{ KB} = \mathbf{64 \text{ MB}}$ - **Total ≈ 64 MB + 256 KB + 10 KB**

Worked 3 — reverse form. Block size 8 KB, pointer 8 bytes, and the maximum file must reach at least 1 TB. How many indirection levels are needed? Pointers per block = $8192/8 = 1024$. - Single: $1024 \times 8 \text{ KB} = 8 \text{ MB}$ - Double: $1024^2 \times 8 \text{ KB} = 8 \text{ GB}$ - Triple: $1024^3 \times 8 \text{ KB} = \mathbf{8 \text{ TB}}$ ✓ **Three levels of indirection** are required.

NUMERICAL TYPE 42c — FAT size

FAT entry width must address every block on the volume: $\lceil \log_2(\text{number of blocks}) \rceil$ bits **FAT size = number of blocks × entry size**

Worked 1. Disk of 4 GB, block size 4 KB. Blocks = $2^{32}/2^{12} = 2^{20} \rightarrow$ entry needs 20 bits $\rightarrow$ round to **3 bytes** (or 4 for alignment). FAT size = $2^{20} \times 3 = \mathbf{3 \text{ MB}}$ (or 4 MB with 4-byte entries).

Worked 2. Disk of 1 TB, block 4 KB. Blocks = $2^{40}/2^{12} = 2^{28} \rightarrow 28 \text{ bits} \rightarrow \mathbf{4 \text{ bytes}}$. FAT = $2^{28} \times 4 = \mathbf{1 \text{ GB}}$. This is why FAT does not scale to large volumes.

Worked 3 — FAT32 naming. FAT32 uses 32-bit entries (28 usable). With 4 KB clusters it addresses $2^{28} \times 4 \text{ KB} = \mathbf{1 \text{ TB}}$.

NUMERICAL TYPE 42d — free space management size

Bit vector size = number of blocks / 8 bytes

Worked 1. Disk 1 GB, block 4 KB. Blocks = $2^{30}/2^{12} = 2^{18} = 262,144 \rightarrow$ bit vector = 262,144 bits = **32 KB**

Worked 2. Disk 40 GB, block 1 KB. Blocks = $40 \times 2^{30} / 2^{10} = 40 \times 2^{20} \approx 41,943,040 \text{ bits} \approx \mathbf{5 \text{ MB}}$

Methods compared

Method	Mechanism	Strength
Bit vector	One bit per block	Easy to find contiguous runs
Linked list	Free blocks chained	No wasted space; traversal slow
Grouping	First free block holds addresses of n free blocks	Finds many free blocks quickly
Counting	Stores (first free block, count) pairs	Compact when free space is clustered

9.5 File system implementation layers

Application $\rightarrow$ logical file system (metadata, directory structure, protection) $\rightarrow$ file-organisation module (logical-to-physical block mapping, free space) $\rightarrow$ basic file system (issues generic block commands) $\rightarrow$ I/O control (device drivers, interrupt handlers) $\rightarrow$ devices.

On-disk structures: boot control block (per volume), volume control block / **superblock** (block count, block size, free block count, free FCB pointers), directory structure, per-file **FCB / inode**. **In-memory structures:** mount table, directory cache, system-wide open file table, per-process open file table, buffer cache.

9.6 Consistency and recovery

- **Consistency checker** (`fsck`, `chkdsk`) compares directory structure with block allocation data.
- **Journaling / log-structured filesystems** write metadata changes to a log first, then apply them; on crash, replay the log. Much faster recovery than a full scan.

PART 10 — I/O SYSTEMS AND DISK SCHEDULING

10.1 I/O transfer methods

Method	CPU involvement	Mechanism
Programmed I/O (polling)	Highest — CPU busy-waits on a status bit	Simple, wasteful
Interrupt-driven I/O	Moderate — CPU interrupted per byte/word	Better utilisation
DMA	Lowest — CPU involved only at setup and completion	Controller transfers blocks directly to memory

DMA modes: - **Burst mode** — the controller holds the bus until the whole block is transferred. Fastest transfer, CPU stalled longest. - **Cycle stealing** — the controller takes **one bus cycle at a time**, interleaving with CPU activity. Balanced. - **Transparent mode** — transfers only when the CPU is not using the bus. No CPU impact, slowest.

Cycle stealing is the one asked by name.

10.2 Interrupts

Vectored — the device supplies an index into the interrupt vector table; dispatch is immediate. **Non-vectored** — control goes to a fixed address which must then poll. **Daisy chaining** resolves priority among devices on a shared line. **Maskable vs non-maskable** interrupts; NMI cannot be disabled.

10.3 Disk structure and geometry

Platter → **surface** → **track** → **sector**. A **cylinder** is the set of tracks at the same radius across all surfaces. **Number of cylinders = tracks per surface**.

Constant Linear Velocity (CLV) — outer tracks hold more sectors; rotation speed varies (CDs). **Constant Angular Velocity (CAV)** — uniform sectors per track; constant rotation (hard disks).

NUMERICAL TYPE 43a — total head movement, six algorithms

Head at cylinder 53. Queue: 98, 183, 37, 122, 14, 124, 65, 67. Disk range 0-199.

FCFS — arrival order

53→98→183→37→122→14→124→65→67 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2 = **640**

SSTF — nearest next

53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183 12 + 2 + 30 + 23 + 84 + 24 + 2 + 59 = **236** Risk: starvation of distant requests.

SCAN — sweep to the disk end, then reverse (head initially moving up)

53 → 65, 67, 98, 122, 124, 183 → **199** → 37, 14 (199 - 53) + (199 - 14) = 146 + 185 = **331**

C-SCAN — always sweeps one direction, jumps back

53 → 199 → (jump to 0) → 14, 37 (199 - 53) + (199 - 0) + (37 - 0) = 146 + 199 + 37 = **382** If the return jump is excluded by convention: 146 + 37 = 183. *State the assumption.*

LOOK — like SCAN but reverses at the last request, not the disk end

53 → 183 → 14 (183 - 53) + (183 - 14) = 130 + 169 = **299**

C-LOOK — like C-SCAN but jumps from the last request back to the first

53 → 183 → (jump to 14) → 37 130 + (183 - 14) + (37 - 14) = 130 + 169 + 23 = **322** Excluding the jump: 130 + 23 = 153.

Summary

Algorithm	Movement
FCFS	640
SSTF	236
LOOK	299
C-LOOK	322
SCAN	331
C-SCAN	382

Three things that change the answer: the **initial direction** of head travel, whether **SCAN or LOOK** is meant (disk edge vs last request), and whether the **return jump** counts. All three are stated in well-written questions — read for them.

NUMERICAL TYPE 43b — disk access time

Access time = Seek time + Rotational latency + Transfer time
Average rotational latency = half a rotation = 30/RPM
seconds
Transfer time = bytes / transfer rate

Worked 1. 7200 RPM, average seek 5 ms, transfer rate 40 MB/s, read 4 KB. - Rotation = $60/7200 = 8.33\text{ ms}$ → **average latency = 4.17 ms** - Transfer = $4096 / 40 \times 10^6 = 0.1\text{ ms}$ - **Total ≈ 9.27 ms**

Worked 2. 10,000 RPM, seek 4 ms, transfer rate 50 MB/s, read 8 KB. - Latency = 3 ms · Transfer = $8192/50 \times 10^6 = 0.16\text{ ms}$ - **Total ≈ 7.16 ms**

Rotational latency reference table — memorise

RPM	One rotation	Average latency
3600	16.67 ms	8.33 ms
5400	11.11 ms	5.56 ms
7200	8.33 ms	4.17 ms
10,000	6 ms	3 ms
12,000	5 ms	2.5 ms
15,000	4 ms	2 ms

NUMERICAL TYPE 43c — disk capacity from geometry

Capacity = surfaces × tracks per surface × sectors per track × bytes per sector

Worked 1. 4 platters (**8 surfaces**), 2000 tracks per surface, 64 sectors per track, 512 bytes per sector. $8 \times 2000 \times 64 \times 512 = 524,288,000\text{ bytes} \approx 500\text{ MB}$

Worked 2. 16 surfaces, 128 tracks/surface, 256 sectors/track, 512 B/sector. $16 \times 128 \times 256 \times 512 = 2^{28} = 256\text{ MB}$

Careful: *n platters means 2n surfaces* (both sides used). Questions give platters and expect you to double. Some state “one surface unused for data” — read it.

Cylinders = tracks per surface = 2000 in worked example 1.

NUMERICAL TYPE 43d — transfer rate from geometry

Transfer rate = bytes per track / rotation time

Worked. 7200 RPM, 512 sectors per track, 512 bytes per sector. - Bytes per track = $512 \times 512 = 262,144$ - Rotation = 8.33 ms - Rate = $262,144 / 0.00833\text{ s} \approx 31.5\text{ MB/s}$

10.4 RAID

Level	Technique	Usable capacity (n disks)	Fault tolerance	Min disks
RAID 0	Striping	n	None	2
RAID 1	Mirroring	n/2	One per mirrored pair	2
RAID 2	Bit-level striping + Hamming ECC	Varies	One	3
RAID 3	Byte-level striping + dedicated parity	n – 1	One	3
RAID 4	Block-level striping + dedicated parity disk	n – 1	One	3
RAID 5	Block striping + distributed parity	n – 1	One	3
RAID 6	Block striping + double distributed parity	n – 2	Two	4
RAID 10 (1+0)	Mirrored then striped	n/2	One per mirror	4

Worked. 6 disks of 2 TB each. - RAID 0 → **12 TB**, no tolerance - RAID 1 → **6 TB**, survives one failure per pair - RAID 5 → $(6-1) \times 2 = 10\text{ TB}$, survives one - RAID 6 → $(6-2) \times 2 = 8\text{ TB}$, survives two - RAID 10 → **6 TB**

RAID 4 vs RAID 5 — RAID 4’s dedicated parity disk becomes a write bottleneck; RAID 5 distributes parity across all disks, removing it. Standard comparison question.

RAID 0 is not redundant despite the name “redundant array”. It is the one level with zero fault tolerance. Very frequently tested.

PART 11 — PROTECTION AND SECURITY

11.1 Protection

Domain of protection — a set of (object, rights) pairs. **Access matrix** — rows are domains, columns are objects. Implemented as an **access control list** (per object) or a **capability list** (per domain).

Principle of least privilege — grant only the rights needed.

UNIX permissions — three triplets (owner, group, others) each with read/write/execute. Octal: r=4, w=2, x=1. So **755** = owner rwx, group r-x, others r-x. **644** = owner rw-, group r-, others r-. Directly asked in bank papers. **setuid bit** — a program runs with the owner's privileges rather than the caller's.

11.2 Security threats

Threat	Mechanism
Virus	Attaches to a host program; needs a carrier and user action
Worm	Self-replicating, self-propagating; needs no host
Trojan horse	Disguised as legitimate software
Logic bomb	Triggers on a condition
Rootkit	Hides its own presence, usually at kernel level
Ransomware	Encrypts data and demands payment
Spyware / keylogger	Covert monitoring
Buffer overflow	Overwrites the stack return address to execute injected code
Denial of service	Exhausts a resource
Man in the middle	Intercepts communication
Phishing	Social engineering for credentials

Virus vs worm is the single most-asked security distinction: a virus needs a host program and user action; a worm propagates on its own.

11.3 Defences

Authentication (passwords, biometrics, multi-factor) · encryption (symmetric and asymmetric) · **ASLR** and **stack canaries** against buffer overflows · firewalls · intrusion detection · auditing and logging · sandboxing · code signing.

PART 12 — MASTER FORMULA SHEET

Processes - n sequential forks $\rightarrow 2^n$ processes, $2^n - 1$ children - Prints inside an n -iteration fork loop (print after fork) $\rightarrow 2^{n+1} - 2$

Scheduling - $TAT = CT - AT - WT = TAT - BT - RT = \text{first CPU time} - AT$ - RR slices = $\sum [BT_i/q]$; context switches = slices - 1 - CPU efficiency = useful time / total elapsed - Throughput = processes / total time - $RR = FCFS$ when $q \geq \text{max burst}$

Synchronization - Semaphore final value = $S_0 - \text{waits} + \text{signals}$; negative magnitude = blocked count - Producer-consumer: mutex = 1, empty = n , full = 0 - Two processes, n non-atomic increments each: max = $2n$, min = 2 - Nested semaphores A and B $\rightarrow$ effective concurrency = $\min(A, B)$

Deadlock - Uniform need: deadlock-free if $m \geq n(R - 1) + 1$ - Non-uniform: deadlock-free if $m \geq \sum (R_i - 1) + 1$ - Max processes: $n \leq (m - 1)/(R - 1)$, **round down** - Need = Max - Allocation; Available = Total - $\sum$ Allocation

Memory - Offset bits = $\log_2(\text{page size})$ - Page number bits = logical bits - offset bits - Frame number bits = $\log_2(\text{physical memory} / \text{frame size})$ - Paging internal fragmentation $\approx$ half a page per process - Segmentation: valid offset range is 0 to limit - 1

Virtual memory - Page table size = (address space / page size) $\times$ PTE size - Entries per page table = page size / PTE size - Multilevel levels = $\lceil \text{page number bits} / \log_2(\text{entries per table}) \rceil$ - Inverted page table entries = number of frames - TLB EAT (hierarchical) = $h(t_TLB + t_mem) + (1-h)(t_TLB + 2 \cdot t_mem)$ - TLB EAT (parallel) = $h \cdot t_mem + (1-h) \cdot 2 \cdot t_mem$ - TLB EAT (L levels) = $h(t_TLB + t_mem) + (1-h)(t_TLB + (L+1) \cdot t_mem)$ - Page fault EAT = $(1-p) \cdot t_mem + p \cdot t_fault$ - Dirty page: average fault cost = $t_fault(1 + d)$ - Hit ratio + fault rate = 1 - Thrashing when $\sum$ working set sizes > available frames

File systems - Disk accesses: contiguous 1 $\cdot$ linked n $\cdot$ indexed 2 $\cdot$ two-level indexed 3 - Inode max size = direct + $(P) + (P^2) + (P^3)$ blocks, where $P = \text{block size} / \text{pointer size}$ - FAT size = blocks $\times$ entry width; entry width $\geq \lceil \log_2(\text{blocks}) \rceil$ bits - Bit vector size = number of blocks / 8 bytes

Disk - Access time = seek + rotational latency + transfer - Average rotational latency = 30/RPM seconds - Capacity = surfaces $\times$ tracks $\times$ sectors $\times$ bytes per sector - n platters = $2n$ surfaces - Cylinders = tracks per surface - Transfer rate = bytes per track / rotation time

RAID - RAID 0 $\rightarrow n$ - RAID 1 $\rightarrow n/2$ - RAID 5 $\rightarrow n-1$ - RAID 6 $\rightarrow n-2$ - RAID 10 $\rightarrow n/2$

PART 13 — COMPLETE TRAP LIST

Fundamentals 1. `printf`, `scanf`, `malloc`, `fopen` are **library functions**, not system calls. 2. **Mode switch $\neq$ context switch**. Every context switch involves mode switches; not conversely. 3. Microkernels are **slower** than monolithic kernels, because of message-passing overhead. 4. Multiprogramming (switches on I/O) $\neq$ multitasking (switches on quantum) $\neq$ multiprocessing (multiple CPUs). 5. `read()` copies **kernel space $\rightarrow$ user space**, not hardware $\rightarrow$ user space.

Processes and threads 6. **Waiting $\rightarrow$ Running is impossible**. Must pass through Ready. 7. **Ready $\rightarrow$ Waiting is impossible**. Must be running to request I/O. 8. **Stack, registers, PC and thread ID are private**; code, data, files and heap are shared. 9. The **long-term scheduler** controls the degree of multiprogramming. 10. In **many-to-one** threading, one blocking call blocks the whole process.

IPC 11. **Anonymous pipes are unidirectional** and need a common ancestor. 12. **A pipe is message passing**, not shared memory — its buffer is shared with the kernel. 13. **Semaphores and signals transfer no data**. 14. Shared memory is the **only** model requiring explicit synchronisation. 15. Pipes have **no message boundaries**; message queues do.

Synchronization 16. **Test-and-set does not provide bounded waiting** without extra bookkeeping. 17. **Strict alternation violates progress**, not mutual exclusion. 18. Producer-consumer: **counting semaphore before mutex**, or it deadlocks. 19. **Readers-writers (reader preference) starves writers**. 20. Naive dining philosophers **deadlocks** via circular wait. 21. **A lost `signal()` on a condition variable has no effect**; a semaphore signal is remembered in the count. 22. **Only a mutex has ownership semantics**; any process can signal a semaphore. 23. Minimum concurrent output is **2, not 1**, for two processes.

Deadlock 24. The third condition is “**no preemption**”. An option reading “preemption” is wrong. 25. **A cycle with multiple resource instances does not imply deadlock**. 26. **Unsafe $\neq$ deadlocked**. Deadlock is a subset of unsafe states. 27. Linux, Windows and UNIX **ignore** deadlock rather than prevent it. 28. In the max-processes formula, **round down**. 29. In Banker’s, **the first process in the table is often not the first in the safe sequence** — compute Need and scan all of them.

Memory 30. **Paging $\rightarrow$ internal** fragmentation; **segmentation $\rightarrow$ external**. Constantly swapped in options. 31. **Fixed partitioning $\rightarrow$ internal**; variable partitioning $\rightarrow$ external. 32. Compaction requires **execution-time** address binding. 33. Best fit accommodates more requests but is **slowest** and leaves tiny holes. 34. **Physical address space may exceed logical** — frame bits can exceed page bits. 35. Valid segment offsets are **0 to limit - 1**; offset = limit traps. 36. Segmentation gives a **two-dimensional** address; paging one-dimensional.

Virtual memory 37. **A page fault is a trap, not an error**. 38. Page fault handling ends by **restarting the faulting instruction**. 39. **Belady’s anomaly is FIFO only**. Never LRU or Optimal (stack algorithms). 40. **LRU is not always better than FIFO** on a specific string — only on average. 41. **Optimal is not implementable** — it is a benchmark. 42. **Hierarchical vs parallel TLB access** change the same numbers (140 vs 120 ns). Read the wording. 43. On a TLB miss with an L-level table, the cost is **L + 1** memory accesses. 44. **Thrashing is fixed by reducing** the degree of multiprogramming; adding processes worsens it. 45. The TLB must be **flushed on context switch** unless entries carry ASIDs. 46. Inverted page tables make **shared memory hard** to represent.

File systems and disk 47. **Contiguous allocation gives the best random access** but suffers external fragmentation and hard growth. 48. **Linked allocation makes mid-file insertion cheapest** and random access worst. 49. **Hard links cannot cross filesystems**; symbolic links can and can dangle. 50. **SCAN goes to the disk edge; LOOK reverses at the last request**. 51. **Initial head direction changes SCAN and C-SCAN answers**. 52. **n platters = 2n surfaces**. 53. **Cylinders = tracks per surface**, not total tracks. 54. **RAID 0 has zero fault tolerance** despite “redundant” in the acronym. 55. **RAID 4’s dedicated parity disk is a write bottleneck**; RAID 5 fixes this by distributing parity. 56. **Average rotational latency is half a rotation**, not a full one.

Security 57. **A virus needs a host program and user action; a worm is self-propagating**. 58. UNIX permission 755 = `rxr-xr-x`; 644 = `rw-r-r-`.

PART 14 — PRACTICE SETS

Set A — theory, 30 questions

1. Which scheduler controls the degree of multiprogramming?
2. Which process state transition is impossible: Ready→Running, Running→Ready, Waiting→Running, Running→Waiting?
3. Which of these is NOT shared between threads of a process: code section, heap, stack, open files?
4. Why is a microkernel slower than a monolithic kernel?
5. Which IPC mechanism requires processes to have a common ancestor?
6. Which IPC mechanism transfers no data?
7. Strict alternation using only a `turn` variable violates which CS requirement?
8. Does test-and-set provide bounded waiting?
9. In producer-consumer, what happens if the producer executes `wait(mutex)` before `wait(empty)`?
0. Which classical synchronization problem starves writers?
 1. Name the fix for priority inversion.
 2. Which is NOT a deadlock condition: mutual exclusion, hold and wait, preemption, circular wait?
 3. Does a cycle in a resource allocation graph always mean deadlock?
 4. Does an unsafe state always mean deadlock?
 5. Which operating systems ignore deadlock entirely?
 6. Which scheduling algorithm is optimal for average waiting time with staggered arrivals?
 7. What is the convoy effect?
 8. What is the standard fix for starvation in priority scheduling?
 9. For what quantum does Round Robin behave identically to FCFS?
0. Which fragmentation type does paging cause, and where?
 1. Which address binding scheme is required for compaction?
 2. A segment has limit 100. Is offset 100 valid?
 3. Is a page fault an error?
 4. What is the last step in page fault handling?
 5. Which page replacement algorithms can exhibit Belady's anomaly?
 6. Why can LRU never exhibit it?
 7. What is the fix for thrashing?
 8. Which file allocation method gives the best random access?
 9. Difference between SCAN and LOOK?
 0. Which RAID level has no fault tolerance?

Set B — numericals, 30 questions

1. `for(i=0;i<4;i++) fork();` — total processes?
2. `fork(); fork(); printf("X");` — how many X printed?
3. `for(i=0;i<3;i++){ fork(); printf("X"); }` — how many X printed?
4. `if(fork()!=0) fork();` — total processes?
5. Semaphore starts at 8; 15 waits, 4 signals. Value and blocked count?
6. Semaphore value is -5 after 20 waits and 8 signals. Initial value?
7. Two processes each execute a non-atomic `x=x+1` twelve times, x starts 0. Min and max final value?
8. Buffer of size 15. Initial values of mutex, empty, full?
9. Six processes, each needing at most 5 units of one resource. Minimum units for guaranteed deadlock freedom?
0. Processes needing maximums 3, 5, 8. Minimum total units?
1. 20 units available; each process needs at most 5. Maximum processes?
2. Processes P1(AT 0, BT 6), P2(AT 2, BT 4), P3(AT 4, BT 2). Average WT under SRTF?
3. Same set, average WT under FCFS?
4. Same set, all arriving at 0 instead, average WT under SJF?
5. Three processes at t=0, bursts 5, 2, 7. RR q=3. Average TAT?
6. Same, number of context switches (excluding the final one)?
7. Three processes at t=0, bursts 4, 6, 2. RR q=2, context switch 1 ms. CPU efficiency?
8. Partitions 200, 400, 600, 500 KB; requests 300, 450, 180 KB. Which partition serves 450 under best fit?
9. Process of 35 KB, page size 8 KB. Internal fragmentation?
0. 40-bit logical address, 16 KB pages. Offset and page number bits?
 1. 2 GB physical memory, 8 KB frames. Frame number bits?
 2. Segment base 5000, limit 400. Physical address for (segment, 399) and (segment, 400)?
 3. 32-bit logical address, 32 segments, 8 KB pages. Three-part bit split?
 4. 40-bit virtual address, 16 KB pages, 8-byte PTE. Single-level page table size?
 5. 32-bit address, 2 KB page, 4-byte PTE. Two-level bit split?
 6. 48-bit address, 8 KB page, 8-byte PTE. Number of levels required?
 7. TLB 15 ns, memory 90 ns, hit ratio 92%, hierarchical. EAT?
 8. Memory 150 ns, fault service 12 ms, fault rate 0.0008. EAT?

- Reference string 3,1,4,1,5,9,2,6,5,3,5 with 3 frames, FIFO and LRU. Faults each?
- Head at 75, requests 42, 110, 18, 95, 130, 60. SSTF total movement?

Set C — file systems and disk, 15 questions

- Linked allocation: accesses to read block 15?
 - Single-level indexed: accesses to read block 15?
 - Inode with 12 direct, 1 single, 1 double, 1 triple indirect. Block 8 KB, pointer 8 B. Maximum file size?
 - Inode with 8 direct, 1 single indirect. Block 2 KB, pointer 4 B. Maximum file size?
 - Disk 8 GB, block 8 KB. Minimum FAT entry width in bits?
 - Disk 2 GB, block 2 KB. Bit vector size for free space?
 - 6 platters, 1500 tracks per surface, 200 sectors per track, 512 B/sector. Capacity?
 - Same disk — how many cylinders?
 - 15,000 RPM, seek 3 ms, transfer rate 100 MB/s, read 16 KB. Access time?
 - 5400 RPM, 400 sectors/track, 512 B/sector. Transfer rate?
 - 8 disks of 3 TB in RAID 5. Usable capacity?
 - Same disks in RAID 6. Usable capacity and tolerance?
 - Same disks in RAID 10. Usable capacity?
 - Which RAID level has a dedicated parity disk bottleneck?
 - Head at 100, requests 30, 140, 70, 180, 20, 160, disk 0–199, moving up. C-SCAN movement including the jump?
-

Answers — Set A

- Long-term (job) scheduler. 2. **Waiting→Running**. 3. **Stack**. 4. Service calls become kernel-mediated message round-trips instead of function calls. 5. Anonymous pipe. 6. Semaphore (and signal, which carries only a number). 7. **Progress**. 8. **No** — mutual exclusion and progress only. 9. Deadlock: the producer holds the mutex while blocked on `empty`, so the consumer cannot free a slot. 10. Readers-writers (reader-preference version). 11. **Priority inheritance** (or priority ceiling). 12. **Preemption** — the condition is *no* preemption. 13. Only with a single instance per resource type. 14. **No** — deadlock is a subset of unsafe states. 15. Linux, Windows, UNIX. 16. **SRTF**. 17. Short processes queued behind a long one under FCFS, inflating average waiting time. 18. **Ageing**. 19. $q \geq$ the largest burst time. 20. Internal, in the last page only. 21. **Execution-time** binding. 22. **No** — valid offsets are 0 to 99. 23. **No** — it is a normal trap. 24. **Restart the faulting instruction**. 25. FIFO (and LFU/MFU in principle). 26. It is a **stack algorithm** — pages in k frames $\subseteq$ pages in $k+1$ frames. 27. **Reduce the degree of multiprogramming**. 28. **Contiguous**. 29. SCAN travels to the disk edge; LOOK reverses at the last pending request. 30. **RAID 0**.

Answers — Set B

- 16** (2^4). 2. **4**. 3. **14** ($2 + 4 + 8$). 4. **3**. 5. $8 - 15 + 4 = -3$, three blocked. 6. $S_0 - 20 + 8 = -5 \rightarrow S_0 = 7$. 7. Max **24**, min **2**. 8. **1, 15, 0**. 9. $6(5-1) + 1 = 25$. 10. $2 + 4 + 7 + 1 = 14$. 11. $4n + 1 \leq 20 \rightarrow n \leq 4.75 \rightarrow 4$. 12. SRTF: P1 0–2, P2 2–6, P3 6–8, P1 8–12. CT = 12, 6, 8. TAT = 12, 4, 4. WT = 6, 0, 2 $\rightarrow$ avg **2.67**. 13. FCFS: P1 0–6, P2 6–10, P3 10–12. TAT = 6, 8, 8. WT = 0, 4, 6 $\rightarrow$ avg **3.33**. 14. SJF at $t=0$, order 2, 4, 6: WT = 0, 2, 6 $\rightarrow$ avg **2.67**. 15. RR $q=3$: P1 0–3, P2 3–5, P3 5–8, P1 8–10, P3 10–14. CT = 10, 5, 14. TAT = 10, 5, 14 $\rightarrow$ avg **9.67**. 16. Slices = $\lceil 5/3 \rceil + \lceil 2/3 \rceil + \lceil 7/3 \rceil = 2+1+3 = 6 \rightarrow$ **5** switches. 17. Useful 12 ms; slices = $2+3+1 = 6 \rightarrow$ 5 switches = 5 ms; total 17 ms $\rightarrow$ **70.6%**. 18. **500 KB** ($300 \rightarrow 400$, so 450 fits best in 500). 19. $\lceil 35/8 \rceil = 5$ pages = 40 KB $\rightarrow$ **5 KB**. 20. Offset **14**, page number **26**. 21. $2^{31}/2^{13} = 2^{18} \rightarrow$ **18 bits**. 22. $\langle 399 \rangle \rightarrow$ **5399**; $\langle 400 \rangle \rightarrow$ **trap**. 23. $5 \mid 14 \mid 13$. 24. Offset 14 $\rightarrow$ pages $2^{26} \rightarrow 2^{26} \times 8 =$ **512 MB**. 25. Offset 11; entries = $2048/4 = 512 = 2^9 \rightarrow$ **12 | 9 | 11**. 26. Offset 13 $\rightarrow$ page bits 35; entries = $1024 = 2^{10} \rightarrow \lceil 35/10 \rceil =$ **4 levels**. 27. $0.92(105) + 0.08(195) = 96.6 + 15.6 =$ **112.2 ns**. 28. $0.9992(150) + 0.0008(12,000,000) = 149.88 + 9,600 =$ **9,749.88 ns**. 29. String 3,1,4,1,5,9,2,6,5,3,5. FIFO 3 frames: **9 faults**; LRU 3 frames: **9 faults** — they coincide on this string. 30. SSTF: $75 \rightarrow 60(15) \rightarrow 42(18) \rightarrow 18(24) \rightarrow 95(77) \rightarrow 110(15) \rightarrow 130(20) =$ **169**.

Answers — Set C

- 15**. 2. **2**. 3. $P = 8192/8 = 1024$. Direct 12×8 KB = 96 KB; single 1024×8 KB = 8 MB; double 8 GB; triple **8 TB** $\rightarrow \approx$ **8 TB**. 4. $P = 512$. Direct 16 KB + single 512×2 KB = 1 MB $\rightarrow \approx$ **1 MB + 16 KB**. 5. Blocks = $2^{33}/2^{13} = 2^{20} \rightarrow$ **20 bits**. 6. Blocks = $2^{31}/2^{11} = 2^{20} \rightarrow 2^{20}$ bits = **128 KB**. 7. 12 surfaces $\times 1500 \times 200 \times 512 =$ **1,843,200,000 bytes** $\approx$ **1.72 GB**. 8. **1500**. 9. Latency 2 ms + seek 3 ms + transfer $16384/100 \times 10^6 = 0.16$ ms $\rightarrow$ **5.16 ms**. 10. Bytes/track = 204,800; rotation 11.11 ms $\rightarrow$ **18.4 MB/s**. 11. $(8-1) \times 3 =$ **21 TB**. 12. $(8-2) \times 3 =$ **18 TB**, survives **two** failures. 13. **12 TB**. 14. **RAID 4**. 15. $100 \rightarrow 140, 160, 180 \rightarrow 199 \rightarrow$ jump to 0 $\rightarrow 20, 30, 70, (199-100) + 199 + 70 =$ **368**.